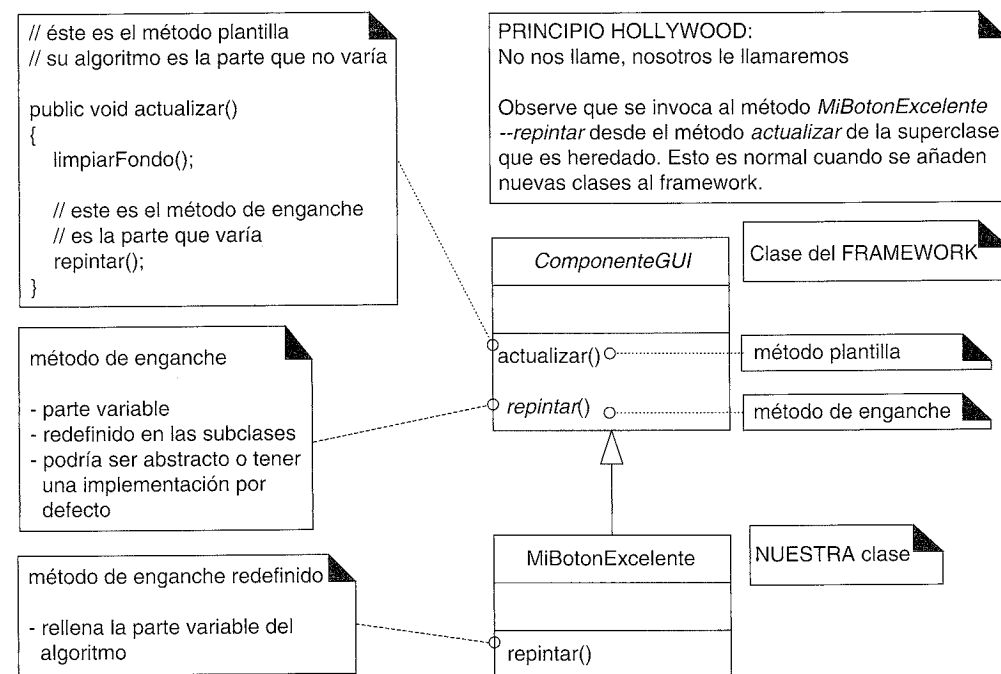


La idea es crear un método (el Método Plantilla) en una superclase que define el esqueleto de un algoritmo, con sus partes variables e invariables. El Método Plantilla invoca otros métodos, algunos de los cuales podrían redefinirse en una subclase. De esta manera, las subclases pueden redefinir los métodos que varían para añadir su propio comportamiento único en los puntos de variabilidad (ver Figura 34.6).



34.12. Materialización con el patrón Método Plantilla

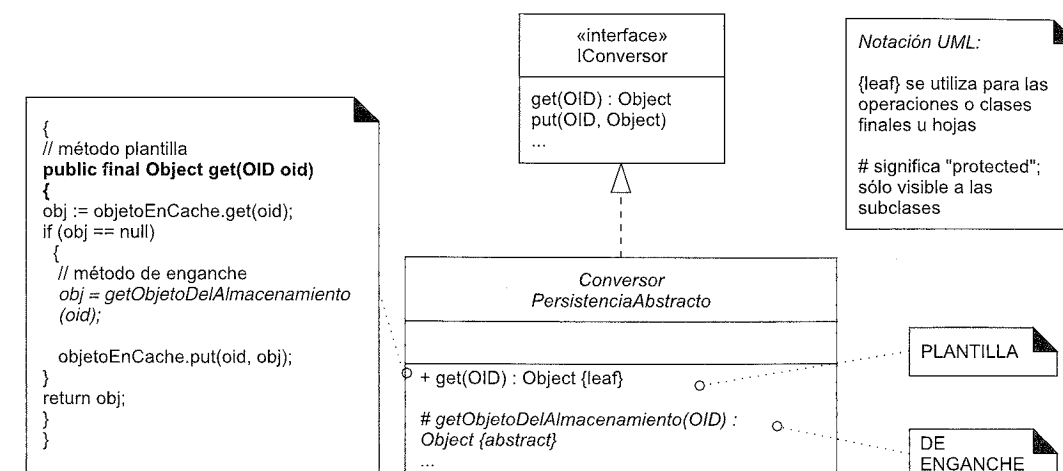
Si tuviéramos que programar dos o tres clases para establecer la correspondencia con el mecanismo de almacenamiento permanente, apreciaríamos partes comunes en el código. La estructura básica que se repite del algoritmo para materializar un objeto es:

```
if (objeto está en cache)
    return obj
else
    crear el objeto a partir de su representación en el almacenamiento
    guardar el objeto en la cache
    return obj
```

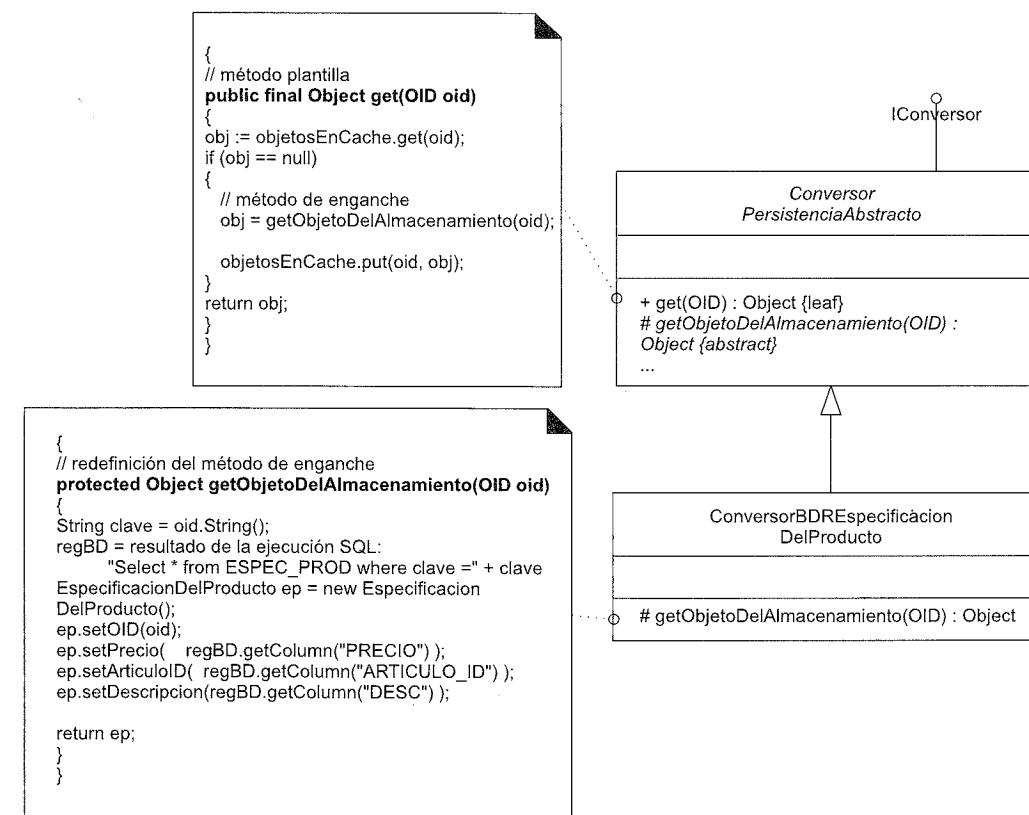
El punto de variación es la manera de crear el objeto a partir del almacenamiento.

Crearemos el método *get* que será el método plantilla en una superclase abstracta *ConversorPersistenciaAbstracto* que define la plantilla, y utiliza un método “de enganche” (*hook method*) en las subclases para la parte que varía. La Figura 34.7 muestra el diseño esencial.

Como se muestra en este ejemplo, es normal que el método plantilla sea *público*, y que el método de enganche sea *protegido*. El *ConversorPersistenciaAbstracto* e *Iconversor* forman parte del FWP. Ahora, un programador de aplicaciones puede incorporar



elementos en este framework añadiendo una subclase, y redefiniendo o implementando el método de enganche *getObjetoDelAlmacenamiento*. La Figura 34.8 muestra un ejemplo.



⁵ En Java por ejemplo, el *regBD* que devuelve la ejecución de la consulta SQL sería un *ResultSet* de JDBC.

Asuma que en la implementación del método de enganche de la Figura 34.8, la primera parte del algoritmo —la ejecución del SELECT de SQL— es la misma para todos los objetos, sólo varía el nombre de la tabla de la base de datos⁶. Si se sostiene esta suposición, entonces una vez más, podría aplicarse el patrón Método Plantilla para factorizar por separado las partes que varían y las que no. En la Figura 34.9, la parte artificiosa es que *ConversorAbstractoBDR*--*getObjetoDelAlmacenamiento* es un método de enganche con respecto al método *ConversorPersistenciaAbstracto*--*get*, pero es un método plantilla con respecto al nuevo método de enganche *getObjetoDelRegistro*.

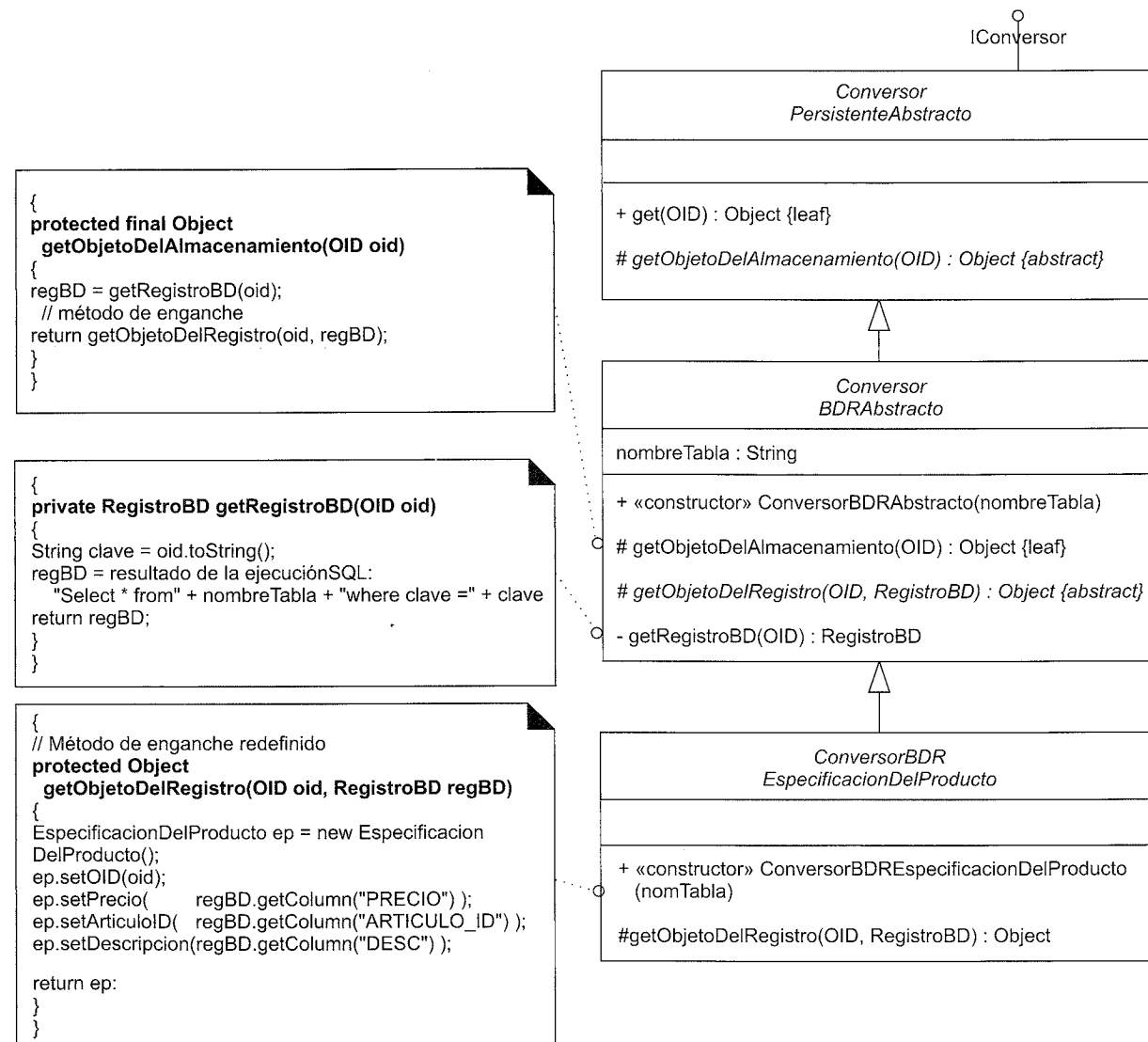


Figura 34.9. Ajustando el código de nuevo con el Método Plantilla.

⁶ En muchos casos, la situación no es tan simple. Un objeto podría derivarse a partir de los datos contenidos en dos o más tablas o a partir de múltiples bases de datos, en cuyo caso, la primera versión del diseño del Método Plantilla es más flexible.

Notación UML: Observe cómo se pueden declarar los constructores en UML. El estereotipo es opcional, y si se utiliza la convención de que el nombre del constructor sea igual al nombre de la clase, probablemente es innecesario.

Ahora *IConversor*, *ConversorPersistenciaAbstracto* y *ConversorBDRAbstracto* forman parte del framework. El programador de la aplicación sólo necesita añadir su subclase, como *ConversorBDREspecificacionDelProducto*, y asegurar que se crea con el nombre de la tabla (se pasa mediante el encadenamiento de constructores hasta el *ConversorBDRAbstracto*).

La jerarquía de clases del Conversor de Base de Datos es una parte esencial del framework; el programador de la aplicación podría añadir nuevas subclases para adaptarlo a nuevos tipos de mecanismos de almacenamiento persistente o nuevas tablas o ficheros específicos en un mecanismo de almacenamiento existente. La Figura 34.10 muestra la estructura de algunos de los paquetes y clases. Nótese que las clases específicas del proyecto NuevaEra no pertenecen al paquete general de servicios técnicos *Persistencia*.

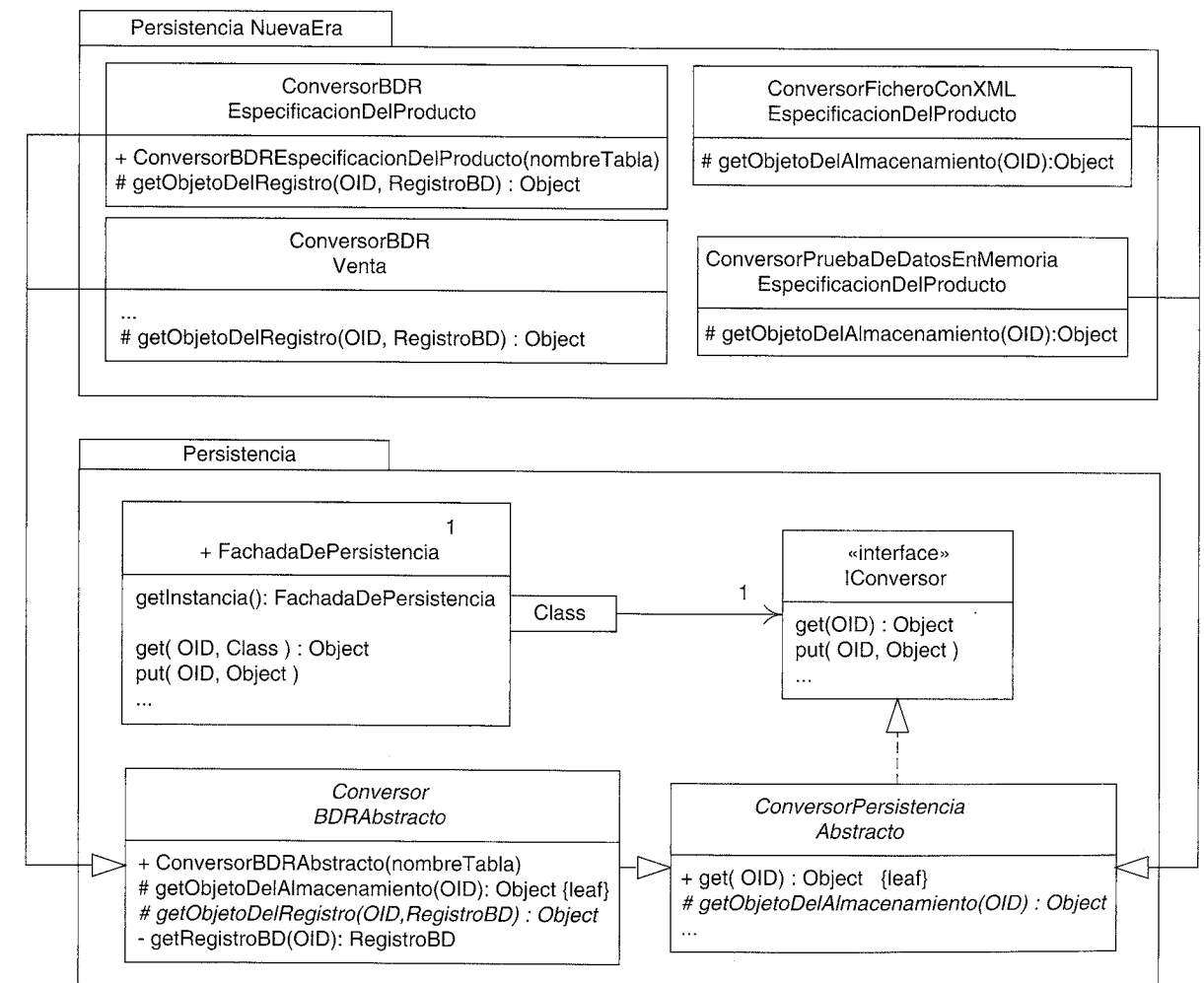


Figura 34.10. El framework de persistencia.

Creo que este diagrama, combinado con la Figura 34.9, ilustra el valor de un lenguaje visual como UML para describir las partes del software; transmite de manera concisa mucha información.

Obsérvese la clase *ConversorPruebaDeDatosEnMemoriaEspecificacionDelProducto*. Tales clases se pueden utilizar para servir objetos con valores definidos directamente en el código para hacer pruebas, sin acceder a ningún almacenamiento persistente externo.

El UP y el Documento de la Arquitectura del Software (SAD)

En cuanto al UP y la documentación, recordemos que el SAD sirve de ayuda para el aprendizaje de futuros desarrolladores, que contiene vistas de la arquitectura con las ideas claves relevantes. La inclusión de diagramas como los de las Figuras 34.9 y 34.10 en el SAD para el proyecto NuevaEra está muy en la línea del tipo de información que un SAD debería contener.

Métodos sincronizados o de guarda en UML

El método *ConversorPersistenciaAbstracto--get* contiene código con secciones críticas que no es seguro en el hilo de ejecución —se podría materializar el mismo objeto de manera concurrente en hilos diferentes—. Como subsistema de servicios técnicos, el servicio de persistencia necesita que se diseñe teniendo presente la seguridad de los hilos. De hecho, el subsistema completo podría estar distribuido en un proceso separado en otro ordenador, transformando la *FachadaDePersistencia* en un objeto servidor remoto, y con muchos hilos ejecutándose simultáneamente en el subsistema, sirviendo a múltiples clientes.

Por tanto, el método debería tener control de la concurrencia de los hilos —si se utiliza Java, se añadirá la palabra clave *synchronized*—. La Figura 34.11 ilustra un método sincronizado en un diagrama de clases.

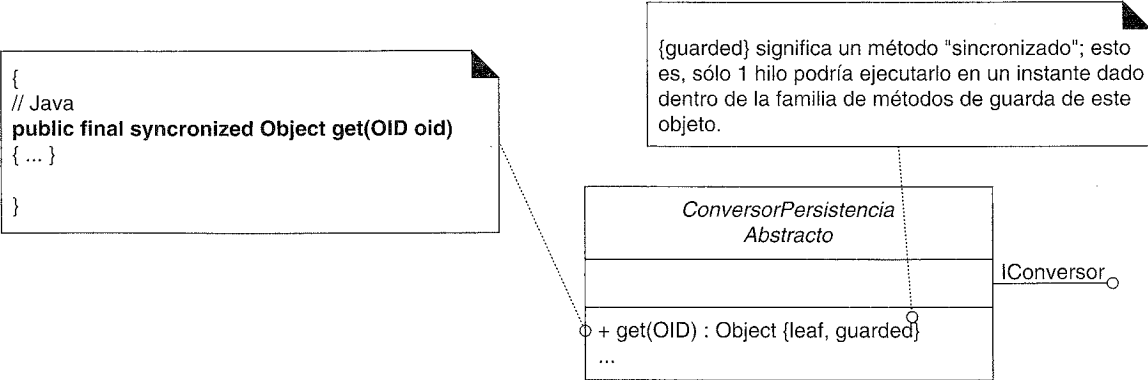


Figura 34.11. Métodos de guarda en UML.

34.13. Configuración de conversores con una FactoriaDeConversores

Análogamente a los ejemplos anteriores de factorías en el caso de estudio, la configuración de la *FachadaDePersistencia* con un conjunto de objetos *IConversor* se puede conseguir con un objeto factoría, *FactoriaDeConversores*. Sin embargo, un pequeño giro, no es conveniente nombrar cada conversor con una operación diferente. Por ejemplo, no es deseable:

```
class FactoriaDeConversores
{
public IConversor getConversorEspecificacionDelProducto() {...}
public IConversor getConversorVenta() {...}
...
}
```

Esto no soporta Variaciones Protegidas con respecto a la lista creciente de conversores —y crecerá—. En consecuencia, es preferible lo siguiente:

```
class FactoriaDeConversores
{
public Map getTodosLosConversores() {...}
...
}
```

donde las claves del *java.util.Map* (probablemente implementado con un *HashMap*) son los objetos *Class* (los tipos persistentes), y los objetos *IConversor* son los valores.

Entonces, la fachada puede inicializar su colección de objetos *IConversor* como sigue:

```
class FachadaDePersistencia
{
private java.util.Map conversores =
FactoriaDeConversores.getInstancia().getTodosLosConversores();
...
}
```

La factoría puede asignar un conjunto de objetos *IConversor* utilizando un diseño dirigido por los datos. Es decir, la factoría puede leer las propiedades del sistema para descubrir qué clases *IConversor* debe instanciar. Si se utiliza un lenguaje con capacidades de programación reflexiva, como Java, entonces la instanciación se puede basar en la lectura de los nombres de las clases como cadenas de texto, y la utilización de algo como la operación *Class.newInstance* para crear las instancias. De esta manera, se puede reconfigurar el conversor sin cambiar el código fuente.

34.14. Patrón: Gestión de Caché

Es conveniente mantener los objetos materializados en una caché local para mejorar el rendimiento (la materialización es relativamente lenta) y dar soporte a las operaciones de gestión de las transacciones como commit.

El patrón **Gestión de Caché** (*Cache Management*) [BW96] propone que el *Conversor de Base de Datos* sea el responsable de mantener esta caché. Si se utiliza un conversor diferente para cada clase de objetos persistentes, cada conversor puede mantener su propia caché.

Cuando se materializan los objetos, se colocan en la caché, con su OID como clave. Las siguientes peticiones al conversor para obtener un objeto provocará que el conversor busque primero en la caché, evitando de esta manera materializaciones innecesarias.

34.15. Reunir y ocultar sentencias SQL en una clase

Sentencias SQL embebidas en diferentes clases conversores no es un pecado terrible, pero se puede mejorar. Suponga que en lugar de eso:

- Existe una única clase Fabricación Pura (y es un singleton) *OperacionesBDR* donde se reúnen todas las operaciones SQL (SELECT, INSERT,...).
- Las clases conversores de BDR colaboran con ella para obtener un registro de BD o conjunto de registros (por ejemplo, *ResultSet*).
- Su interfaz es algo parecido a lo siguiente:

```
class OperacionesBDR
{
public ResultSet getDatosEspecificacionDelProducto(OID oid){...}
public ResultSet getDatosVenta(OID oid) {...}
...
}
```

De manera que, por ejemplo, un conversor tiene código como éste:

```
class ConversorBDREspecificacionDelProducto extends ConversorPersistenciaAbstracto
{
protected Object getObjetoDelAlmacenamiento(OID oid)
{
ResultSet rs = OperacionesBDR.getInstancia().getDatosEspecificacionDelProducto(oid);

EspecificacionDelProducto ep = new EspecificacionDelProducto();
ep.setPrecio(rs.getDouble("PRECIO"));
ep.setOID(oid);
return ep;
}
```

A partir de esta Fabricación Pura se obtienen los siguientes beneficios:

- Se facilita el mantenimiento y rendimiento que es ajustado por un experto. La optimización del SQL requiere un programador SQL, en lugar de un programador de objetos. Con todo el SQL embebido en esta única clase, facilita al programador en SQL encontrarlo y trabajar con él.
- Encapsulación de los métodos y detalles de acceso. Por ejemplo, SQL construido mediante código podría sustituirse por una llamada a un procedimiento almacenado en la BDR para obtener los datos. O se podría insertar un enfoque más sofisticado basado en los **metadatos** para generar el SQL, en el que se genera el SQL de manera dinámica a partir de la descripción del esquema de los metadatos que se lee de una fuente externa.

Como arquitecto, el aspecto interesante de esta decisión de diseño es que en ella influyen las habilidades del desarrollador. Se llegó a un compromiso entre alta cohesión y la comodidad de un especialista. No todas las decisiones de diseño están motivadas por cuestiones de ingeniería del software “puras” como el acoplamiento y la cohesión.

34.16. Estados transaccionales y el patrón Estado

Las cuestiones relacionadas con el soporte de las transacciones pueden complicarse, pero para mantener las cosas simples de momento —para centrarnos en el patrón GoF Estado— asuma lo siguiente:

- Los objetos persistentes pueden insertarse, eliminarse o modificarse.
- Operar sobre un objeto persistente (por ejemplo, modificarlo) no provoca una actualización inmediata de la base de datos; más bien se debe ejecutar una operación *commit* explícita.

Además, la respuesta a una operación depende del estado de la transacción del objeto. Como ejemplo, las respuestas se podrían mostrar en la máquina de estados de la Figura 34.12.

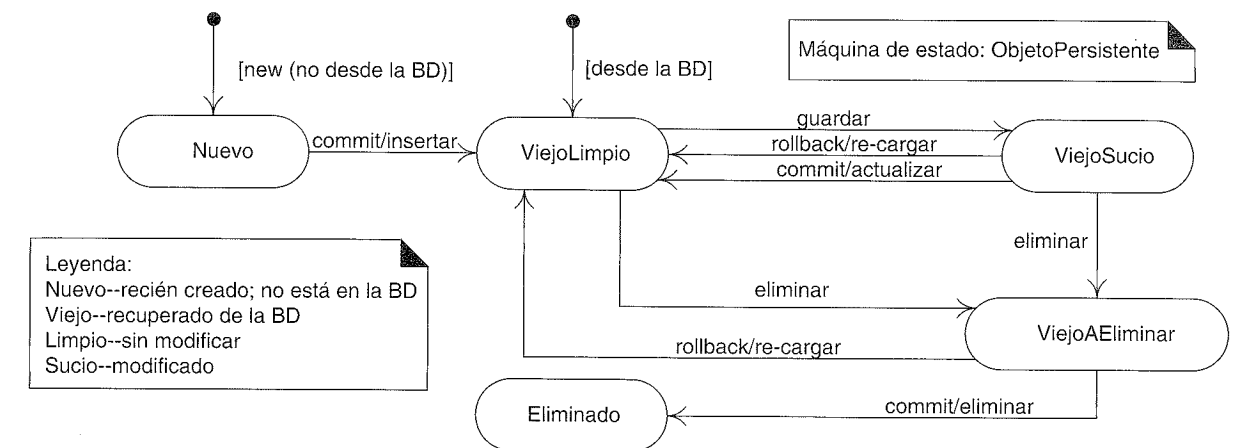


Figura 34.12. Máquina de estados para ObjetoPersistente.

Por ejemplo, un objeto “viejo sucio” (*old dirty*) es aquél recuperado de la base de datos y modificado después. En una operación *commit*, debería actualizarse en la base de datos —a diferencia del que se encuentra en el estado “viejo limpio”, con el que no debería hacer nada (porque no ha cambiado)—. En el FWP orientado a objetos, cuando se ejecuta una operación eliminar o guardar, no origina que se elimine o se guarde inmediatamente en la base de datos; sino, las transiciones de los objetos persistentes al estado apropiado, esperando una operación *commit* o *rollback* para realmente hacer algo.

Como comentario UML, esto es un buen ejemplo de donde es útil una máquina de estados para transmitir información concisa que de otra manera sería difícil de expresar.

En este diseño, asuma que haremos que todas las clases de objetos persistentes extenderán la clase *ObjetoPersistente*⁷, que proporciona los servicios técnicos comunes para la persistencia⁸. Por ejemplo, véase la Figura 34.13.

⁷ [Ambler00b] es una buena referencia acerca de la clase *ObjetoPersistente* y las capas de persistencia, aunque la idea es más antigua.

⁸ Algunas cuestiones relacionadas con la extensión de la clase *ObjetoPersistente* se discuten más tarde. Siempre que una clase de objetos del dominio extienda una clase de los servicios técnicos, se debería hacer una pausa para reflexionar, ya que se está mezclando intereses de la arquitectura (persistencia y lógica de la aplicación).

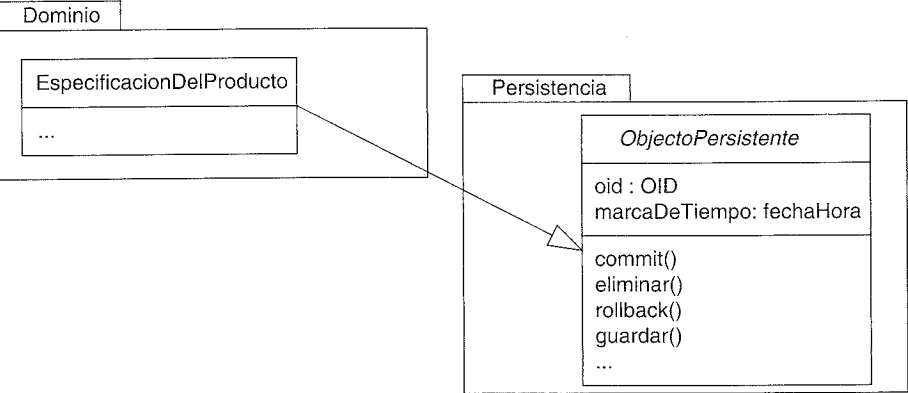


Figura 34.13. Objetos persistentes.

Ahora —y ésta es la cuestión que se resolverá con el patrón Estado— observe que los métodos *commit* y *rollback* requieren una estructura similar de lógica de casos, basada en el código del estado de la transacción. *commit* y *rollback* ejecutan diferentes acciones en cada caso, pero tienen estructuras lógicas similares.

```
public void commit()
{
    switch(estado)
    {
        case VIEJO_SUCIO:
            //...
            break;
        case VIEJO_LIMPIO:
            //...
            break;
        ...
    }
}
```

```
public void rollback()
{
    switch(estado)
    {
        case VIEJO_SUCIO:
            //...
            break;
        case VIEJO_LIMPIO:
            //...
            break;
        ...
    }
}
```

Una alternativa a esta estructura lógica de casos que se repite es el patrón GoF Estado (*State*).

Estado (*State*)

Contexto/Problema

El comportamiento de un objeto depende de su estado, y sus métodos contienen la lógica de casos que reflejan las acciones condicionales dependientes del estado. ¿Existe una alternativa a la lógica condicional?

Solución

Cree clases estado para cada estado, que implementan una interfaz común. En lugar de definir en el objeto de contexto las operaciones que dependen del estado, deléguelas en su objeto del estado actual. Asegure que el objeto de contexto siempre referencie al objeto estado que refleja su estado actual.

La Figura 34.14 ilustra su aplicación en el subsistema de persistencia.

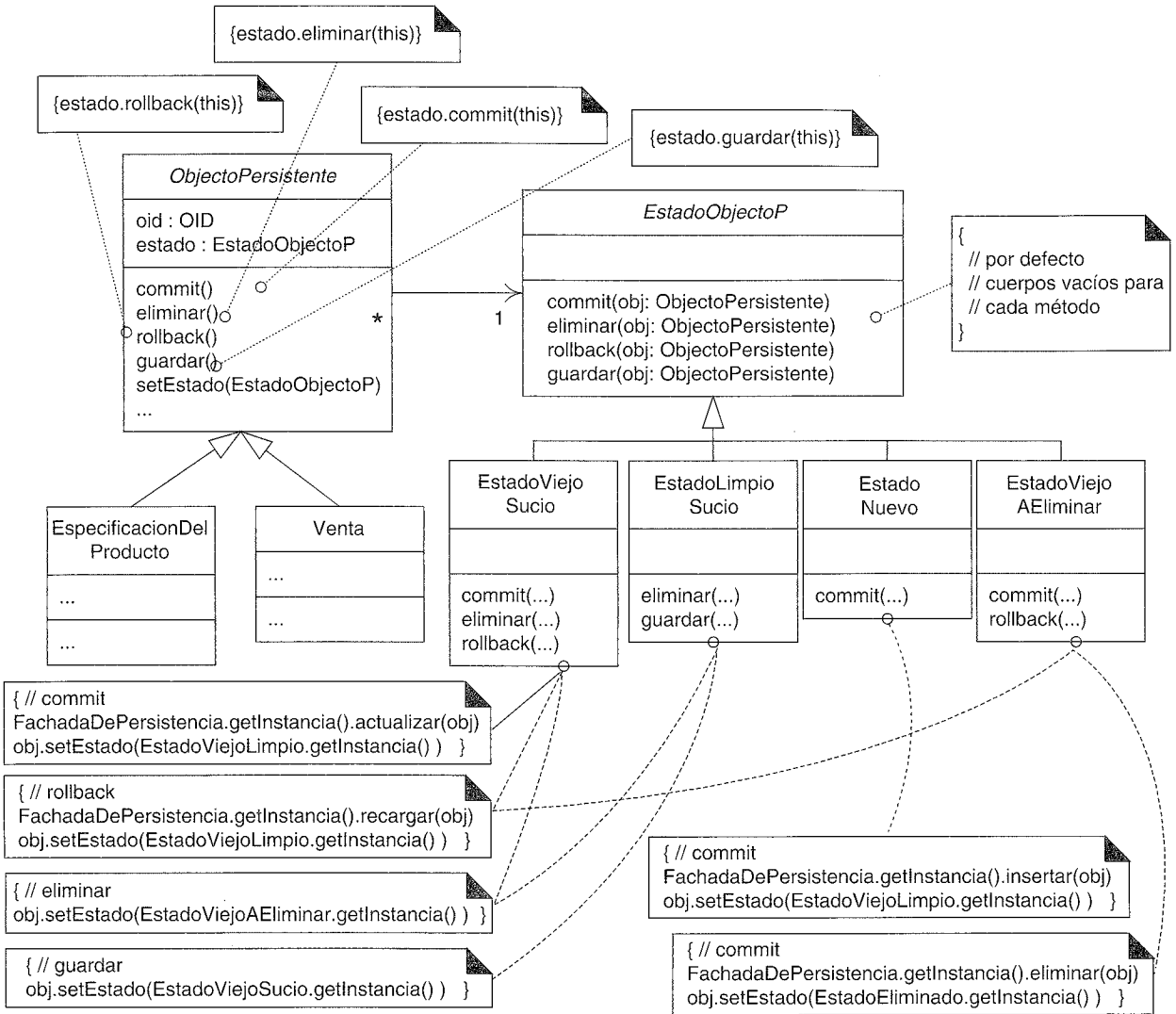


Figura 34.14. Aplicación del patrón Estado⁹.

Los métodos dependientes del estado del *ObjetoPersistente* delegan su ejecución a un objeto estado asociado. Si el objeto de contexto referencia a *EstadoViejoSucio* entonces 1) el método *commit* provocará una actualización de la base de datos, y 2) el objeto de contexto pasará a referenciar a *EstadoViejoLimpio*. Por otro lado, si el objeto de contexto está referenciando a *EstadoViejoLimpio*, se ejecuta el método *commit* heredado como que no hace nada y no hace nada (como se esperaba, puesto que el objeto está limpio).

Observe en la Figura 34.14 que las clases estado y su comportamiento se corresponden con la máquina de estados de la Figura 34.12. El patrón Estado es un mecanismo

⁹ La clase *Borrado* se ha omitido por restricciones de espacio en el diagrama.

para implementar un modelo de transición de estados en software¹⁰. Esto da lugar a que se produzca una transición de un objeto a estados diferentes en respuesta a los eventos.

Un comentario acerca del rendimiento, estos objetos estado no tienen —irónicamente— estado (sin atributos). Por tanto, no es necesario que haya múltiples instancias de una clase —cada una es un singleton—. Miles de objetos persistentes pueden referenciar a la misma instancia *EstadoViejoSucio*, por ejemplo.

34.17. Diseño de una transacción con el Patrón Command

La última sección consideró una vista simplificada de las transacciones. Esta sección amplía la discusión, pero no cubre todas las cuestiones relacionadas con el diseño de las transacciones. Informalmente, una transacción es una unidad de trabajo —un conjunto de tareas— cuyas tareas deben completarse todas con éxito, o no se debe completar ninguna. Es decir, la terminación es atómica.

En cuanto a los servicios de persistencia, las tareas de una transacción incluyen la inserción, actualización y eliminación de los objetos. Una transacción podría contener dos inserciones, una actualización y tres eliminaciones, por ejemplo. Para representar esto, se añade una clase *Transaccion* [Ambler00b]¹¹. Como se señala en [Fowler01], el orden de las tareas de la base de datos dentro de una transacción puede influir en su éxito (y rendimiento).

Por ejemplo:

1. Suponga que la base de datos tiene una restricción de integridad referencial de manera que cuando se actualiza un registro en la TablaA que contiene una clave ajena a un registro de la TablaB, la base de datos requiere que el registro de la TablaB ya exista.
2. Suponga que una transacción contiene una tarea INSERT para añadir el registro de la TablaB, y una tarea UPDATE para actualizar el registro de la TablaA. Si se ejecuta UPDATE antes de INSERT, surge un error de integridad referencial.

La ordenación de las tareas de la base de datos puede ayudar. Algunas cuestiones de la ordenación son específicas del esquema, pero una estrategia general es primero realizar las inserciones, luego las actualizaciones y entonces las eliminaciones.

Tenga en cuenta que el orden en el que una aplicación añade las tareas a una transacción podría no reflejar el mejor orden de ejecución. Las tareas necesitan que se ordenen justo antes de su ejecución.

Esto nos lleva a otro patrón GoF: Command¹².

¹⁰ Existen otros, entre los que se encuentran lógica condicional construida mediante código, intérpretes de máquinas de estados, y generadores de código dirigidos por tablas de estados.

¹¹ Se denomina UnidadDeTrabajo en [Fowler01].

¹² N. del T. No se ha traducido por el mismo motivo que en el caso del patrón Singleton.

Contexto/Problema

¿Cómo gestionar las solicitudes o tareas que necesitan funciones como ordenar (estableciendo prioridades), poner en cola, retrasar, anotar en registro o deshacer?

Solución

Defina una clase por cada tarea que implemente una interfaz común.

Éste es un patrón sencillo con muchas aplicaciones útiles; las acciones se convierten en objetos, y de esta manera se pueden ordenar, anotar en un registro, poner en cola, etcétera. Por ejemplo, en el FWP, la Figura 34.15 muestra las clases Command (o tareas) para las operaciones de la base de datos.

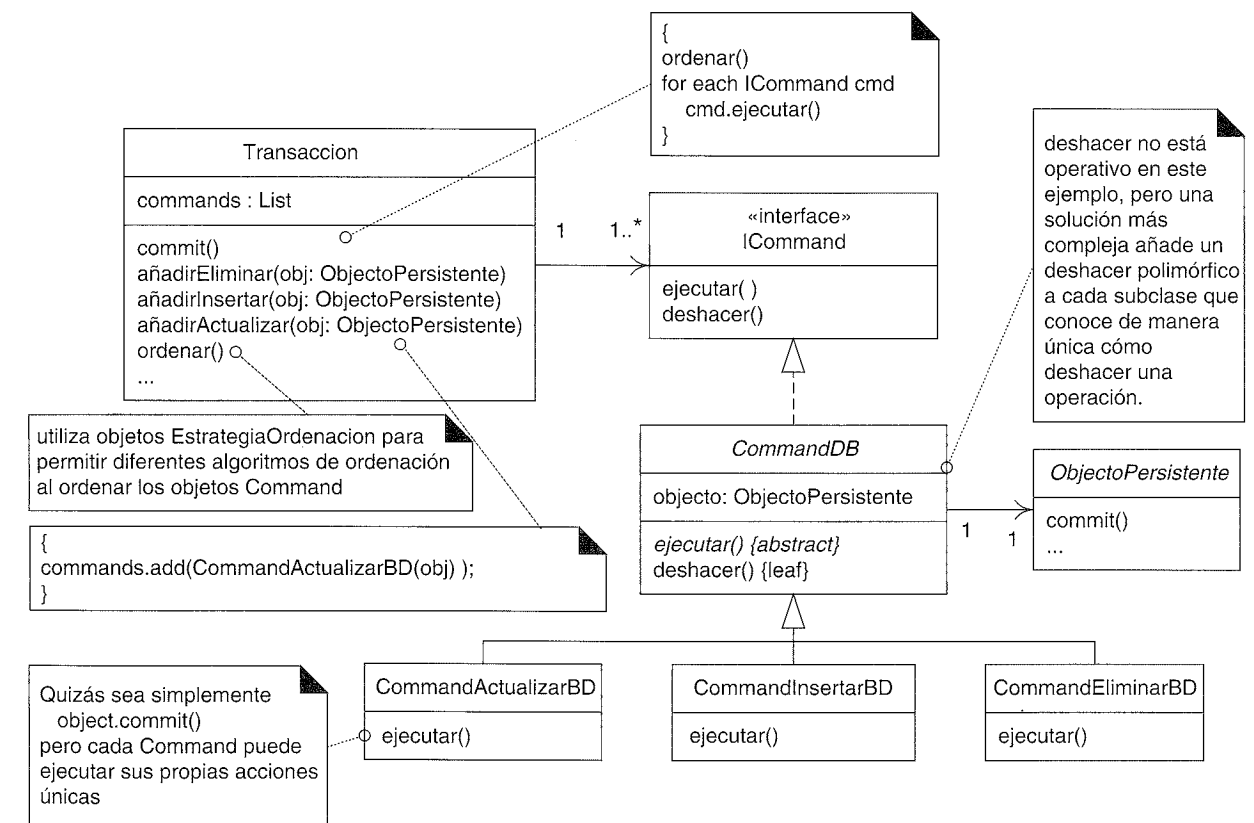


Figura 34.15. Clases Command para las operaciones de la base de datos.

Hay mucho más para completar una solución de transacción, pero la idea clave de esta sección es representar cada tarea o acción de la transacción como un objeto con un método *ejecutar* polimórfico; esto hace accesible un mundo de flexibilidad tratando la respuesta como un objeto en sí.

El ejemplo prototípico de Command es el de las acciones de una GUI, como cortar y pegar. Por ejemplo, el método *ejecutar* de *CommandCortar* realiza la acción cortar, y su

método *deshacer* deshace el corte. *CommandCortar* también retendrá los datos necesarios para llevar a cabo la acción de deshacer. Todos los objetos *Command* de la GUI se pueden mantener en una pila que recoge la historia de las ejecuciones, de manera que se puedan desapilar por turnos, y deshacerse cada una.

Otro uso típico del patrón *Command* es para la gestión de las peticiones del lado del servidor. Cuando un objeto servidor recibe un mensaje (remoto), crea un objeto *Command* para esta petición, y lo entrega al *CommandProcesador* [BMRSS96], que puede poner en cola, anotar en un registro, priorizar y ejecutar los objetos *Command*.

34.18. Materialización perezosa con un Proxy Virtual

Algunas veces es conveniente diferir la materialización de los objetos hasta que sea absolutamente necesario, normalmente por razones de rendimiento. Por ejemplo, suponga que los objetos *EspecificacionDelProducto* referencian a un objeto *Fabricante*, pero rara vez es necesario que se materialice desde la base de datos. Sólo provocan una petición de la información del fabricante escenarios poco frecuentes, como los escenarios relacionados con las rebajas de los fabricantes en el que se necesita el nombre y la dirección de la compañía.

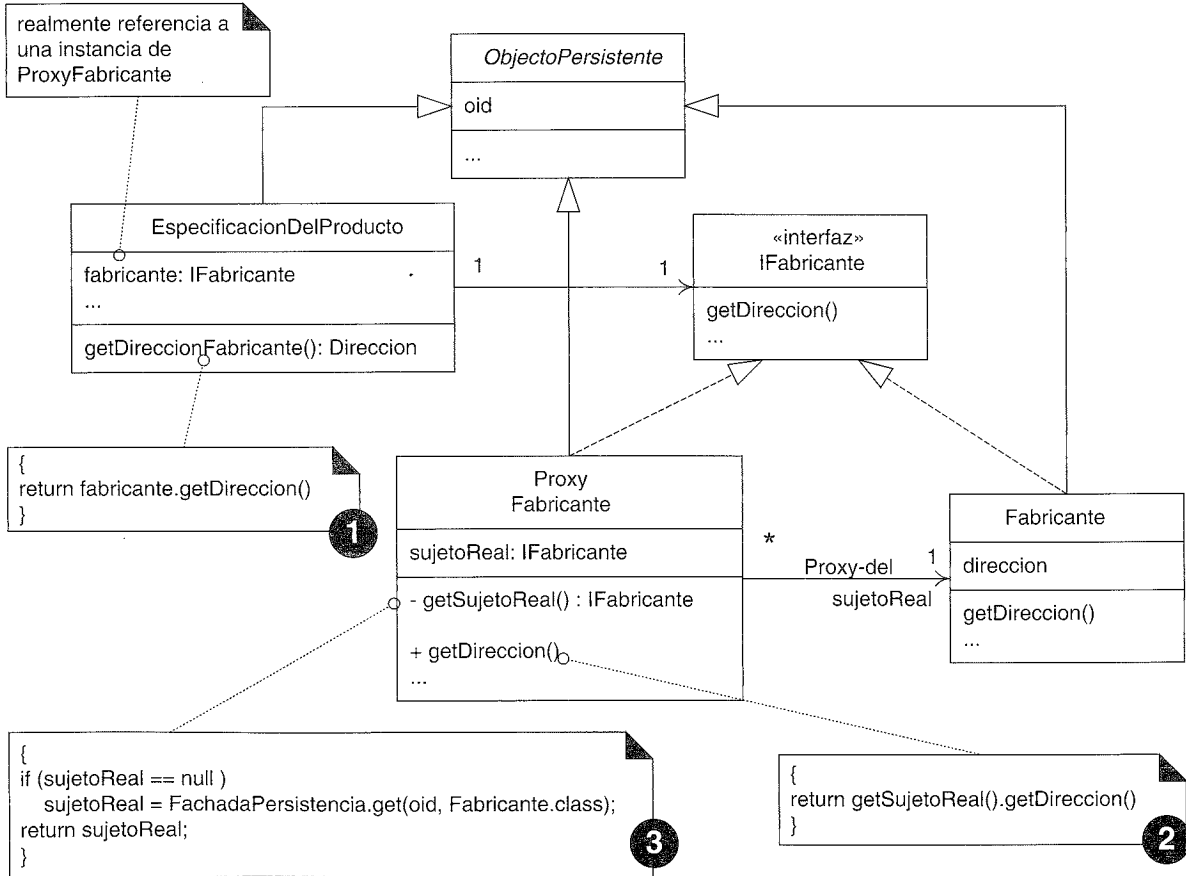


Figura 34.16. Proxy Virtual del Fabricante.

La materialización diferida de los objetos “hijos” se conoce como **materialización perezosa**. La materialización perezosa se puede implementar utilizando el patrón *GoF Proxy Virtual* —una de las muchas variaciones del *Proxy*—.

Un **Proxy Virtual** es un proxy para otro objeto (el *sujeto al que se quiere acceder realmente*) que materializa el sujeto real cuando se referencia por primera vez; por tanto, implementa la materialización perezosa. Es un objeto ligero que simboliza el objeto “real” que puede o no ser materializado.

Un ejemplo concreto del patrón *Proxy Virtual* con la *EspecificacionDelProducto* y el *Fabricante* se muestra en la Figura 34.16. Este diseño se basa en la suposición de que los proxies conocen el *OID* de sus sujetos reales, y cuando se requiere la materialización, se utiliza el *OID* para ayudar a identificar y recuperar el sujeto real.

Nótese que la *EspecificacionDelProducto* tiene visibilidad de atributo de la instancia *IFabricante*. El *Fabricante* para esta *EspecificacionDelProducto* podría no haberse materializado todavía en memoria. Cuando la *EspecificacionDelProducto* envía un mensaje *getDireccion* al *ProxyFabricante* (creyendo que es el objeto fabricante materializado), el proxy materializa el *Fabricante* real, utilizando el *OID* del *Fabricante* para recuperarlo y materializarlo.

Implementación de un Proxy Virtual

La implementación de un *Proxy Virtual* varía según el lenguaje. Los detalles quedan fuera del alcance de este capítulo, pero a continuación presentamos un resumen:

Lenguaje	Implementación del Proxy Virtual
C++	Define una clase plantilla de puntero inteligente (<i>smart pointer</i>). Realmente no se necesita la definición de la interfaz <i>IFabricante</i> .
Java	Se implementa la clase <i>ProxyFabricante</i> . Se define la interfaz <i>IFabricante</i> . Sin embargo, normalmente no se codifican manualmente. Más bien, uno crea un generador de código que analiza las clases a las que se quiere acceder realmente (ej. <i>Fabricante</i>) y genera <i>IFabricante</i> y <i>ProxyFabricante</i> . Otra alternativa de Java es utilizar el API <i>Proxy Dinámico</i> .
Smalltalk	Definir un <i>Proxy Morphing Virtual</i> (o <i>Proxy Fantasma</i>), que utiliza <i>#doesNotUnderstand</i> : y <i>#become</i> : para transformar en el sujeto real. No se necesita la definición de <i>IFabricante</i> .

¿Quién crea el Proxy Virtual?

Observe en la Figura 34.16 que el *ProxyFabricante* colabora con la *FachadaDePersistencia* para materializar su sujeto real. ¿Pero quién crea el *ProxyFabricante*? Respuesta: la clase que establece la correspondencia entre la *EspecificacionDelProducto* y la base de datos. Esta clase es la responsable de decidir, cuándo se materializa un objeto, cuál de los objetos “hijos” debería también materializarse de manera impaciente, y cuáles deberían materializarse de manera perezosa con un proxy.

Considere estas soluciones alternativas: una utiliza materialización impaciente, y la otra materialización perezosa.

//MATERIALIZACIÓN IMPACIENTE DEL FABRICANTE

```
class ConversorBDRespecificacionDelProducto extends ConversorPersistenciaAbstracto
{
protected Object getObjetoDelAlmacenamiento(OID oid)
{
ResultSet rs =
OperacionesBDR.getInstancia().getDatosEspecificacionDelProducto(oid);

EspecificacionDelProducto ep = new EspecificacionDelProducto();
ep.setPrecio(rs.getDouble("PRECIO"));

//aquí está la esencia

String claveAjenaFabricante = rs.getString("FAB_OID");
OID oidFab = new OID(claveAjenaFabricante);
ep.setFabricante((IFabricante)
FachadaDePersistencia.getInstancia().get(oidFab, Fabricante.class);
...
}
```

A continuación presentamos la solución de materialización perezosa:

//MATERIALIZACIÓN PEREZOSA DEL FABRICANTE

```
class ConversorBDRespecificacionDelProducto extends ConversorPersistenciaAbstracto
{
protected Object getObjetoDelAlmacenamiento(OID oid)
{
ResultSet rs =
OperacionesBDR.getInstancia().getDatosEspecificacionDelProducto(oid);

EspecificacionDelProducto ep = new EspecificacionDelProducto();
ep.setPrecio(rs.getDouble("PRECIO"));

//aquí está la esencia

String claveAjenaFabricante = rs.getString("FAB_OID");
OID oidFab = new OID(claveAjenaFabricante);
ep.setFabricante(new ProxyFabricante(oidFab));
...
}
```

34.19. Cómo representar las relaciones en tablas

El código de la sección anterior confía en la clave ajena FAB_OID en la tabla ESPEC_PRODUCTO para conectar con un registro de la tabla FABRICANTE. Esto pone de relieve la pregunta: ¿cómo se representan las relaciones de los objetos en el modelo relacional?

La respuesta se da en el patrón **Representación de las Relaciones de los Objetos en Tablas** [BW96], que propone lo siguiente:

- Asociaciones **uno-a-uno**
 - Colocar una clave ajena OID en una o ambas tablas que representan los objetos de la relación.
 - O, crear una tabla asociación que recoja los OIDs de cada uno de los objetos de la relación.
- Asociaciones **uno-a-muchos**, como una colección
 - Crear una tabla asociativa que registre los OIDs de cada uno de los objetos de la asociación.
- Asociaciones **muchos-a-muchos**
 - Crear una tabla asociativa que registre los OIDs de cada uno de los objetos de la asociación.

34.20. Superclase ObjetoPersistente y separación de intereses

Una solución de diseño parcial típica para proporcionar persistencia a los objetos es crear una clase abstracta de servicios técnicos *ObjetoPersistente* de la que heredan todos los objetos persistentes (ver Figura 34.17). Tal clase, normalmente define atributos para la persistencia, como un OID único, y métodos para almacenarlos en la base de datos.

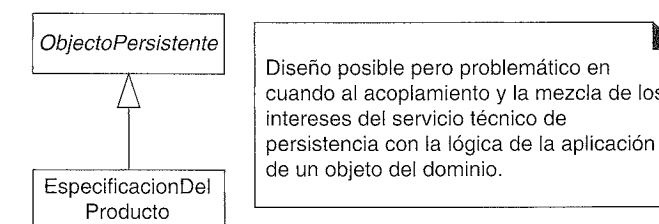


Figura 34.17. Problemas con la superclase ObjetoPersistente.

Esto no es incorrecto, pero adolece de la debilidad del acoplamiento entre la clase y la clase *ObjetoPersistente* —las clases del dominio terminan extendiendo una clase de los servicios técnicos—.

Este diseño no ilustra una clara separación de intereses. Más bien, se mezclan los intereses de los servicios técnicos con los intereses de la lógica del negocio de la capa del dominio en virtud de su extensión.

Por otro lado, la “separación de intereses” no es una virtud absoluta que se debe seguir a cualquier precio. Como se discutió cuando se presentaron las Variaciones Protegidas, los diseñadores necesitan escoger sus batallas en los puntos en los que realmente sea probable una inestabilidad costosa. Si en una aplicación particular, haciendo que la clase sea una subclase de *ObjetoPersistente* nos lleva a una solución ordenada y fácil y no da lugar a problemas de diseño o mantenimiento a largo plazo, ¿por qué no? La respuesta está en entender la evolución de los requisitos y el diseño de la aplicación. También influye el lenguaje: aquéllos con herencia simple (como Java) *consumen* su única preciada superclase.

34.21. Cuestiones sin resolver

Esto ha sido una breve introducción a los problemas y soluciones de diseño de un framework y servicio de persistencia. Se han encubierto muchas cuestiones importantes, entre las que se encuentran:

- Desmaterialización de los objetos.
 - Brevemente, los conversores deben definir los métodos *putObjetoEnAlmacenamiento*. La desmaterialización de jerarquías de composición requiere la colaboración entre múltiples conversores y el mantenimiento de tablas asociativas (si se utiliza un BDR).
- Materialización y desmaterialización de las colecciones.
- Consultas a grupos de objetos.
- Gestión completa de transacciones.
- Gestión de los errores cuando falla una operación de la base de datos.
- Acceso multiusuario y estrategias de bloqueo.
- Seguridad—control del acceso a la base de datos.

SOBRE EL DIBUJO DE DIAGRAMAS Y LAS HERRAMIENTAS

Las burbujas no explotan.
Bertrand Meyer

Objetivos

- Aprender consejos para dibujar los diagramas UML en un proyecto.
 - Ilustrar algunas funciones comunes de las herramientas CASE para UML.
-

Introducción

En un proyecto real, la realización del análisis y diseño mientras se dibujan los diagramas UML no ocurre ordenadamente como en las páginas de un libro. Tiene lugar en el contexto de un equipo de desarrollo de software ocupado, que trabaja en oficinas o habitaciones, hacen garabatos en pizarras y quizás utilizando una herramienta, y a menudo con tendencia a querer comenzar a programar en lugar de trabajar con detalle algunos detalles mediante la realización de diagramas. Si la herramienta para UML, o el proceso para dibujar los diagramas, resultan molestos y engorrosos, o parece que tiene menos valor que la programación, se evitarán.

Este capítulo propone mantener un equilibrio entre la programación y el dibujo de diagramas, y fomentar un entorno de soporte para hacer que la tarea de dibujar los diagramas sea cómoda y útil en lugar de difícil.

35.1. Diseño especulativo y razonamiento visual

Los diseños que se ilustran mediante diagramas UML serán incompletos, y sólo servirán como “trampolín” para la programación. Demasiados diagramas antes de programar nos lleva a una pérdida de tiempo en direcciones de diseño especulativas, o una pérdida de

tiempo obsesionados con herramientas para UML. No hay nada como el código real para decirte qué funciona. Bertrand Meyer lo dice mejor: “Las burbujas no explotan”.

No obstante, promuevo enérgicamente que se anticipe *alguna* idea mediante la realización de diagramas antes de programar, y se que puede ser útil, especialmente para explorar las estrategias de diseño más importantes. La pregunta interesante es “¿cuánto tiempo se tiene que dedicar a dibujar los diagramas antes de programar?” En parte la respuesta está en función de la experiencia y estilo cognitivo de los diseñadores.

Algunas personas son buenas para el razonamiento espacial/visual y expresar sus pensamientos acerca del diseño del software en un lenguaje visual complementa su naturaleza; otros no. Un gran porcentaje del cerebro está dedicado al razonamiento y procesamiento visual o simbólico, en lugar de al procesamiento textual (código). Los lenguajes visuales como UML juegan con una capacidad natural de la mente en la mayoría de la gente. A aquellos a los que se les ha enseñado UML obviamente les resultará más sencillo que a los que no. Y, en general, los diseñadores de objetos con más experiencia pueden diseñar de manera efectiva dibujando sin perderse en especulaciones no realistas, debido a su experiencia y juicio. Aplicados por expertos, los diagramas pueden ayudar a un grupo a moverse más rápidamente hacia un diseño apropiado, debido a la capacidad de ignorar los detalles y centrarse en los verdaderos problemas.

Una excepción a esta sugerencia de dibujar los diagramas “ligeros” son los sistemas que se modelan de manera natural como máquinas de estados. Hay algunas herramientas CASE que pueden hacer un trabajo impresionante generando código completo basado en máquinas de estados UML detalladas para todas las clases. Pero no todos los dominios encajan de manera natural en un enfoque fuertemente centrado en las máquinas de estados; por ejemplo, las máquinas de control y telecomunicaciones a menudo se ajustan bien, los sistemas de información de gestión normalmente no.

35.2. Sugerencias para dibujar los diagramas de UML en el proceso de desarrollo

Nivel de esfuerzo

Como guía, considere realizar los diagramas en parejas durante los siguientes periodos, antes de programar de manera seria en la iteración.

Iteración de 2-semanas	De medio día a un día casi al comienzo de la iteración (ej. lunes o martes)
Iteración de 4-semanas	Uno o dos días cerca del comienzo

En ambos casos, la realización de los diagramas no tiene que parar después de este esfuerzo inicial enfocado a este fin, los desarrolladores podrían dirigirse —idealmente en parejas— “a la pizarra” durante sesiones cortas para esbozar ideas antes de programar más. Y podrían realizar otra sesión más larga de medio día a mitad de la iteración, cuando tropiecen con un problema complejo en el ámbito de su tarea inicial, o terminen su primera tarea y pasen a la segunda.

Otras sugerencias

- Dibuje en parejas, no a solas. Lo más importante, la sinergia nos conduce a diseños mejores. En segundo lugar, la pareja aprende rápidamente técnicas de diseño el uno del otro y, por tanto, ambos llegan a ser mejores diseñadores. Es difícil mejorar como diseñador de software cuando uno diseña individualmente. Cambie de manera regular el compañero de dibujo/diseño para estar periodos de tiempo amplios expuestos al conocimiento de los demás.
- Para aclarar un punto al que se ha hecho alusión varias veces, en los procesos iterativos (como el UP), los programadores son también los diseñadores; no hay un equipo separado que dibuja los diseños y se los entrega a los programadores. Los desarrolladores se ponen su sombrero de UML, y dibujan un poco. Entonces se ponen su sombrero de programador e implementan, y continúan diseñando mientras programan.
- Si hay diez desarrolladores, suponga que hay cinco equipos de dibujo trabajando durante un día en pizarras diferentes. Si el arquitecto dedica tiempo a rotar por los cinco equipos, llegará a ver puntos de dependencia, conflictos e ideas de un equipo que sirven para otro. El arquitecto entonces puede actuar de enlace para llegar a una armonía entre los diseños y clarificar las dependencias.
- Contrate un escritor técnico para el proyecto y enséñele algo de la notación UML y los conceptos básicos del A/DOO (de manera que pueda entender el contexto). Disponga de la ayuda del escritor en la realización del “trabajo engorroso” con las herramientas CASE para UML, en los procesos de ingeniería inversa generando los diagramas a partir del código, en la impresión y presentación de grandes diagramas impresos con plotter, etcétera. Los desarrolladores dedicarán su tiempo (más caro) a hacer lo que hacen mejor: idear diseños y programar. Un escritor técnico les ayuda realizando la gestión de los diagramas, además de las verdaderas responsabilidades de la escritura técnica tales como el trabajo en el documento para el usuario final. Esto se conoce como el patrón Analista Mercenario [Coplien95a].
- Organice el área de desarrollo con muchas pizarras amplias distribuidas muy cerca unas de otras.
- Generalizando, maximice el entorno de trabajo para dibujar cómodamente en las paredes. Cree un entorno “amigable” para dibujar y colgar los diagramas. No puede esperar que se consiga una cultura de modelado visual con éxito en un entorno donde los desarrolladores se están peleando por dibujar en pizarras pequeñas de 60x90 cm, monitores de ordenador de tamaño corriente o trozos de papel. Para dibujar de manera cómoda hacen falta espacios muy amplios y abiertos —físicos o virtuales—.
- Como accesorio de las pizarras, utilice finas hojas blancas de plástico con “adherencia estática” (vienen en paquetes de 20 o más) que se pueden colocar en las paredes; éstas se encuentran disponibles en muchas papelerías. Las hojas permanecen unidas a la pared mediante adherencia estática, y se puede utilizar como una pizarra con un rotulador que se pueda borrar. Se puede empapelar una pared con estas hojas para crear “pizarras” temporales, masivas. He dirigido grupos donde

hemos empapelado todas las paredes —de arriba abajo— de la sala del proyecto con estas hojas, y lo encontramos de gran ayuda para la comunicación.

- Si utiliza una pizarra para los dibujos de UML, utilice un dispositivo (existe al menos uno en el mercado) que capture los dibujos hechos a mano y los transmita a un ordenador como un fichero gráfico. Un diseño involucra una parte receptora en la esquina de la pizarra que captura la imagen para enviarla al ordenador y unas fundas especiales transmisoras en las que se insertan los rotuladores.
- De manera alternativa, si utiliza una pizarra para los dibujos de UML, utilice una cámara digital para capturar las imágenes, normalmente en dos o tres secciones. Ésta es una práctica para dibujar los diagramas, bastante común y efectiva.
- Otra tecnología de pizarra es una pizarra “que imprime”, que normalmente es una pizarra de dos lados con un escáner y una impresora conectada. También son útiles.
- Imprima las imágenes UML dibujadas a mano (capturadas mediante una cámara o un dispositivo de pizarra) y cuélguelas de manera visible *muy* cerca de las estaciones de trabajo para la programación. Lo importante de los diagramas es inspirar la dirección de la programación, de manera que los programadores puedan echarles un vistazo mientras están programando. Si se dibujan pero “se entierran”, no tenía mucho sentido dibujarlos.
- Si dibuja los diagramas UML a mano, utilice una notación simple elegida para acelerar y facilitar la realización de los diagramas.
- Incluso si lleva a cabo el diseño creativo en una pizarra, utilice una herramienta CASE para UML para generar los diagramas de paquetes y de clases mediante un proceso de ingeniería inversa a partir del código fuente (de la última iteración) por lo menos al comienzo de cada iteración posterior. Entonces, utilice estos diagramas generados mediante un proceso de ingeniería inversa como punto de partida para el siguiente diseño creativo.
- Imprima periódicamente los diagramas de paquetes y de clases interesantes/ines- tables/difíciles, generados recientemente mediante un proceso de ingeniería in- versa, en un tamaño ampliado (para facilitar que se vea) con un plotter que pueda imprimir en papel continuo de de 90 ó 120 cm de ancho. Cuélguelos en las paredes muy cerca de los desarrolladores como ayuda visual. El escritor técnico, si está presente, puede hacer este trabajo. Anime a los desarrolladores a que dibujen y ha- gan garabatos sobre los diagramas durante el trabajo de diseño creativo.
- Con respecto a la ingeniería inversa, pocas herramientas para UML soportan el proceso de ingeniería inversa para generar diagramas de secuencia —no única- mente diagramas de clases— a partir del código fuente. Si está disponible, utilice una para generar los diagramas de secuencia para los escenarios significativos para la arquitectura, imprímalos a tamaño grande con el plotter, y cuélguelos para fa- cilitar que se vean.
- Si utiliza una herramienta CASE para UML (de hecho, hágalo para todo el trabajo de programación), utilice una estación de trabajo con un monitor dual (dos moni- tores de pantalla plana de tamaño ordinario son más baratas que un único monitor

de pantalla plana de tamaño grande). Los sistemas operativos modernos soportan (al menos) tarjetas de vídeo duales y de esta manera dos monitores. Organice sus ventanas dentro de la herramienta para UML entre los dos monitores. ¿Por qué? Un monitor pequeño reprime psicológicamente y creativamente en lo que se refiere a los dibujos y lenguajes visuales porque el espacio del área visual es demasiado pe- queño y estrecho. Un desarrollador puede caer en la aptitud desmotivada de “el di- seño ha terminado porque la ventana está llena, y parece demasiado desordenado”.

- Cuando utilice una herramienta CASE para UML y realice el diseño creativo por parejas o pequeños grupos, conecte dos proyectores de ordenador a las dos tarjetas de vídeo del ordenador y alinee las proyecciones en la pared de manera que el equipo pueda ver y trabajar con un espacio amplio para el área visual. Un área pe- queña y unos diagramas difíciles de ver son impedimentos psicológicos y sociales para el diseño visual en colaboración de los pequeños grupos.

35.3. Herramientas y características de ejemplo

Este libro es imparcial respecto a la herramienta

Sería algo extraño no mencionar ninguna herramienta CASE (*Computer Aid Software Engineering*, ingeniería del software asistida por ordenador) para UML, porque este li- bro trata en parte de la realización de dibujos en UML, que tiene lugar con una herra- mienta CASE, o en una pizarra. Al mismo tiempo, no se pueden cubrir por igual todas las herramientas, y las evaluaciones apropiadas quedan fuera del alcance de este libro. Para ser imparcial:

Este libro no avala ninguna herramienta CASE para UML. Los siguientes ejemplos sólo sir- ven para ilustrar algunas de las características típicas y claves que podemos encontrar en las herramientas CASE para UML.

Las herramientas se ajustan de manera inconsistente a UML

Pocas herramientas dibujan toda la notación de UML correctamente, y conforme con la versión actual de la especificación de UML —realmente con cualquier versión—. Aun- que esto estaría bien, no debería ser un factor en la elección de una herramienta, puesto que es mucho más importante su funcionalidad y facilidad de uso.

Ejemplo uno

En las Figuras 35.1 y 35.2 se utiliza Together de TogetherSoft para ilustrar y definir dos funciones claves de una herramienta CASE para UML: **ingeniería directa e inversa**. Es- tas funciones son lo que distinguen esencialmente una herramienta CASE para UML de una herramienta para dibujar.

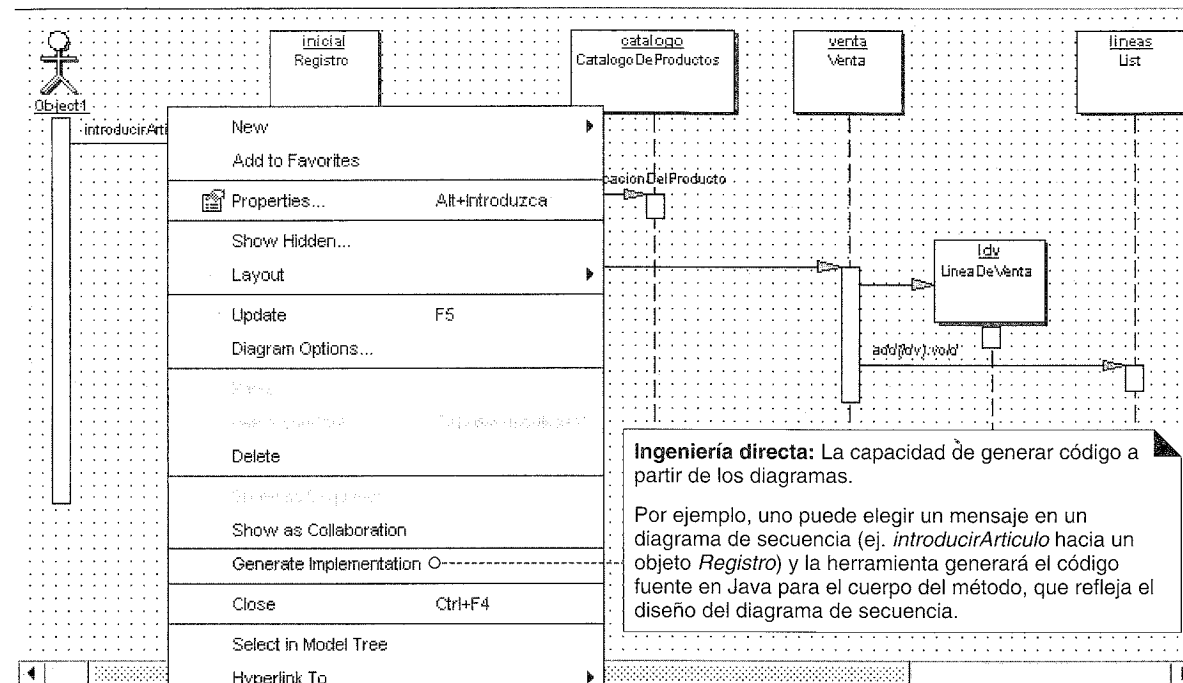


Figura 35.1. Ingeniería directa.

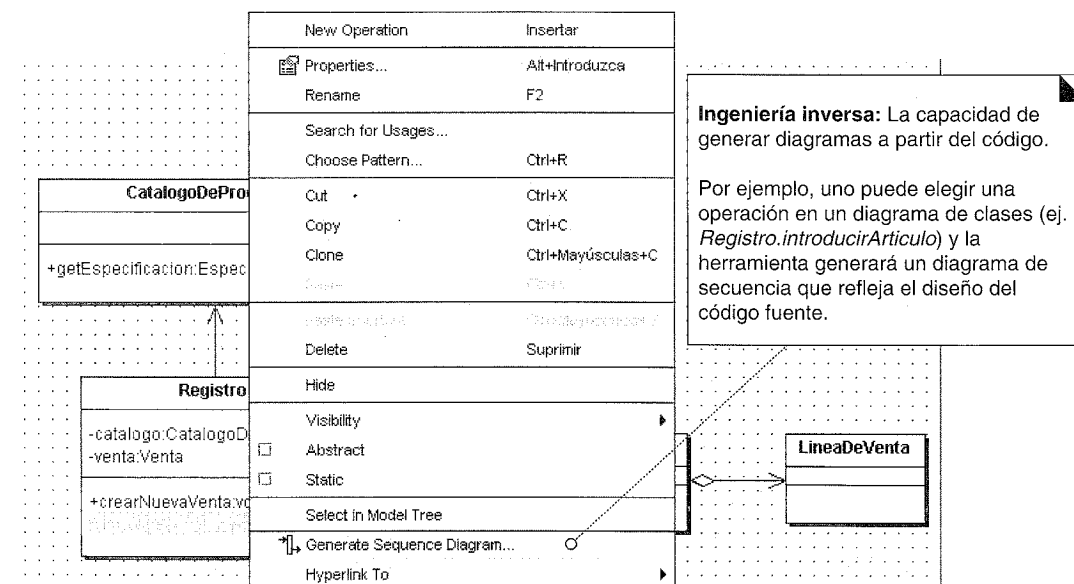


Figura 35.2. Ingeniería inversa.

35.4. Ejemplo dos

En las Figuras 35.3 y 35.4 se utiliza Rational Rose para ilustrar algunas otras funciones básicas de una herramienta CASE para UML.

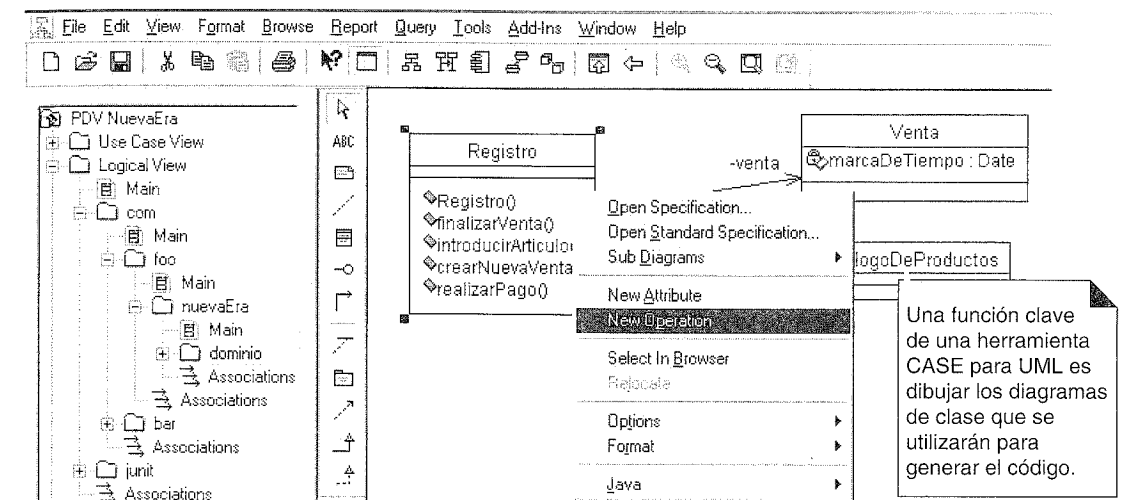


Figura 35.3. Creación de diagramas de clase.

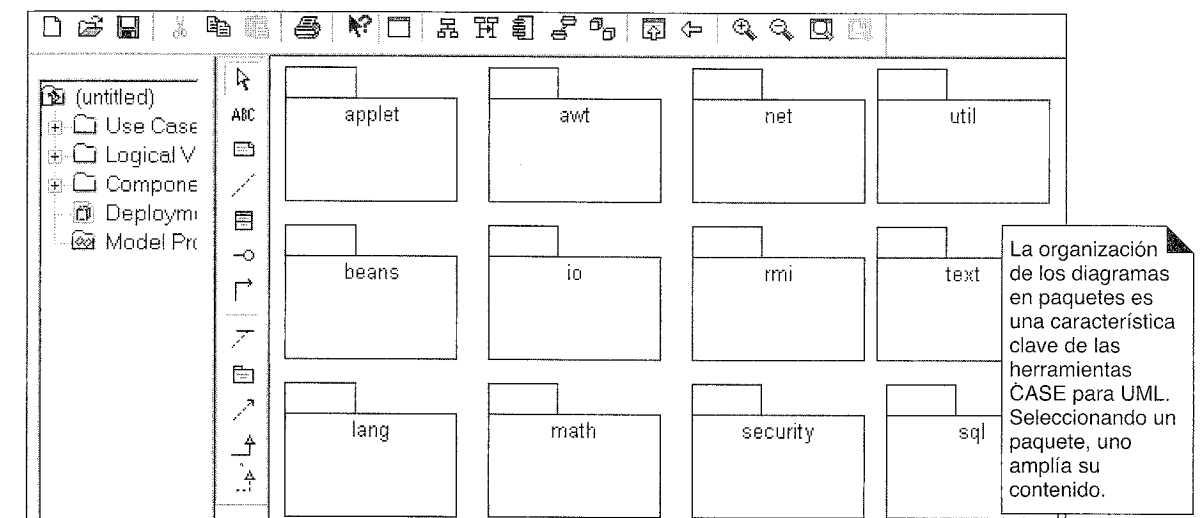


Figura 35.4. Gestión de paquetes.

Peticiones a los vendedores de herramientas CASE para UML

Sugiero que los consumidores hagan cuatro peticiones a los vendedores de herramientas CASE para UML:

1. Implementación correcta de la notación actual de UML en la herramienta.
2. Que el propio equipo de desarrollo de la herramienta CASE haya dibujado, leído y revisado seriamente diagramas UML (incluyendo el proceso de ingeniería inversa de los diagramas) en el proceso de construcción de la propia herramienta para UML.

- 3. Utilizar la versión N de la herramienta para UML para crear la versión N+1.
- 4. Proporcionar el soporte para los procesos de ingeniería directa e inversa de los diagramas de secuencia; la mayoría de las herramientas sólo lo soportan para el diagrama de clases.

Microsoft aboga por que los creadores de las herramientas “coman su propia comida para perros”. Buen consejo.

INTRODUCCIÓN A CUESTIONES RELACIONADAS CON LA PLANIFICACIÓN ITERATIVA Y EL PROYECTO

La predicción es muy difícil, especialmente si es acerca del futuro.
anónimo

Objetivos

- Priorizar los requisitos y los riesgos.
 - Comparar y contrastar la planificación adaptable y predictiva.
 - Definir el Plan de Fase y Plan de Iteración del UP.
 - Introducir las herramientas para mantener la traza de los requisitos para el desarrollo iterativo.
 - Sugerir el modo de organizar los artefactos del proyecto.
-

Introducción

Las cuestiones relacionadas con la gestión y planificación del proyecto son temas amplios, pero sirve de ayuda abordar un breve estudio sobre algunas preguntas claves relacionadas con el desarrollo iterativo y el UP, como:

- ¿Qué hacemos en la siguiente iteración?
- ¿Cómo mantener la traza de los requisitos en el desarrollo iterativo?
- ¿Cómo organizamos los artefactos del proyecto?

36.1. Priorización de los requisitos

Criterios que dirigen las primeras iteraciones: riesgo, cobertura, naturaleza crítica, desarrollo de habilidades

¿Qué hacemos en las primeras iteraciones? Organice los requisitos y las iteraciones de acuerdo con el riesgo, cobertura y naturaleza crítica [Kruchten00]. El riesgo del requisito comprende tanto la complejidad técnica como otros factores, como incertidumbre del esfuerzo, especificación pobre, problemas políticos, o facilidad de uso. Debemos diferenciar la priorización de los riesgos de los requisitos de la priorización de los riesgos del proyecto, que se estudiará en una sección posterior.

La cobertura implica que todas las partes importantes del sistema por lo menos se han tratado brevemente en las primeras iteraciones —quizás una implementación “en anchura y superficial” a través de muchos componentes—. La naturaleza crítica se refiere a las funciones de alto valor para el negocio; es decir, las funciones principales deberían tener al menos implementaciones parciales para los escenarios principales de éxito en las primeras iteraciones, incluso si no son técnicamente arriesgadas.

En algunos proyectos, otro criterio es el desarrollo de habilidades —un objetivo es ayudar al equipo a dominar nuevas habilidades como adoptar las tecnologías de objetos—. En tales proyectos, el desarrollo de habilidades es un factor de gran peso para establecer las prioridades, que tiende a reorganizar las iteraciones en requisitos de menos riesgo o más simples en las primeras iteraciones, motivado por el aprendizaje en lugar de por objetivos de reducción del riesgo.

¿Qué priorizamos?

El UP está dirigido por casos de uso, que incluye la práctica de priorizar los casos de uso (y escenarios de casos de uso) para la implementación. También se expresan algunos requisitos como características de alto nivel no relacionadas con un caso de uso específico, normalmente porque abarcan muchos casos de uso o son un servicio general, como los servicios de registro. Estas funciones que no pertenecen a ningún caso de uso se recogerán en la Especificación Complementaria. Por tanto, incluya en una lista de priorización tanto los casos de uso como las características de alto nivel.

Requisito	Tipo	...
Procesar Venta	CU	...
Registrar	Característica	...
...	...	...

Métodos cualitativos de grupo para la priorización

En base a los anteriores criterios, se priorizan los requisitos, y los de alta prioridad se manejan en las primeras iteraciones. La priorización podría ser informal y cualitativa, generada en una reunión de grupo por miembros conscientes de estos criterios.

Sugerencia

Para priorizar informalmente los requisitos, tareas, o riesgos por medio de una reunión de grupo, utilice la “votación con puntos” de manera iterativa. Liste los ítems en una pizarra. Cada uno coge, por ejemplo, 20 puntos adhesivos. Como grupo, y en silencio (para reducir la influencia), todos se aproximan a la pizarra y aplican los puntos a los ítems, reflejando las prioridades del que vota. En cuanto se termine, se ordenan y discuten. Entonces haga una segunda vuelta de votación con puntos silenciosa para reflejar las percepciones actualizadas en base a la votación de la primera vuelta y a la discusión. Esta segunda vuelta proporciona la retroalimentación y adaptación según la cual se mejoran las decisiones.

La priorización de los requisitos o el riesgo se hará antes de la iteración 1, pero se repetirá de nuevo antes de la iteración 2, y así sucesivamente.

Métodos cuantitativos para la priorización

La discusión de grupo y algo como la votación con puntos para priorizar los requisitos o el riesgo probablemente son suficientes —un enfoque cualitativo difuso—. Para los de mente más cuantitativa, se han utilizado variaciones sobre lo siguiente. Los valores y pesos del ejemplo son sólo sugerencias; lo importante es que los valores numéricos y los pesos se pueden utilizar para razonar acerca de las prioridades.

Requisito	Tipo	SA	Riesgo	Naturaleza crítica	Suma P.
Procesar Venta	CU	3	2	3	15
Registrar	Caract	3	0	1	7
Gestionar Devoluciones	CU	1	0	0	2
...	...	...	...	...	...

	Peso	Rango
SA:Significativo para la Arquitectura	2	0-3
Riesgo: tecnol. complejo, nuevo,...	3	0-3
Naturaleza crítica: valor alto negocio inicial	1	0-3

En cualquier proyecto, los valores exactos no deberían tomarse con demasiada seriedad; en cuanto se termine, se pueden utilizar las puntuaciones numéricas para ayudar a agrupar los requisitos en conjuntos difusos de prioridad alta, media y baja. Claramente, parece importante trabajar en el caso de uso *Procesar Venta* en las primeras iteraciones.

Los números no lo dicen todo. Aunque registrar es una característica simple, de bajo riesgo, es significativo para la arquitectura porque tiene que integrarse completamente en el código desde el principio. Sería difícil y disminuye la integridad de la arquitectura el añadirlo a posteriori.

Priorización de los requisitos del PDV NuevaEra

En base a algunos métodos para priorizar, es posible establecer grupos difusos de requisitos. En términos de los artefactos del UP, esta clasificación se recoge en el Plan de Desarrollo del Software del UP.

Prioridad	Requisitos (caso de uso o característica)	Comentario
Alta	Procesar Venta Registrar ...	Puntuación alta en todos los criterios de clasificación. Generalizado. Difícil de añadir tarde. ...
Media	Gestionar Usuarios Autenticar Usuarios ...	Influye en el subdominio de seguridad. Proceso importante pero no demasiado difícil. ...
Baja	Cerrar Caja Desconectar ...	Fácil; efecto mínimo en la arquitectura. idem. ...

Los casos de uso “Poner en Marcha” y “Desconectar”

Prácticamente todos los sistemas tienen un caso de uso *Poner en Marcha*, implícito si no está explícito. Aunque su prioridad podría no ser alta de acuerdo a otros criterios, es necesario abordar por lo menos una visión simplificada de *Poner en Marcha* en la primera iteración de manera que se proporcione la incialización que asumen otros casos de uso. En cada iteración, se desarrolla incrementalmente el caso de uso *Poner en Marcha* para satisfacer las necesidades de comienzo de los otros casos de uso. De manera análoga, los sistemas a menudo tienen un caso de uso *Desconectar*. En algunos sistemas, es bastante complejo, como desconectar un conmutador de telecomunicaciones activo. En términos de la planificación, si son sencillos, estos casos de uso se pueden listar informalmente en el Plan de Iteración, como “implementar la puesta en marcha y desconexión como se necesite”. Obviamente, versiones más complejas necesitan que se cuiden más los requisitos y la planificación.

Advertencia: Planificación del proyecto vs. objetivos de aprendizaje

El objetivo del libro es ofrecer una ayuda para el aprendizaje de una introducción al análisis y diseño, en lugar de que funcione realmente el proyecto del PDV NuevaEra. Por tanto, se han tomado algunas licencias en la elección de lo que se aborda en las primeras iteraciones del caso de estudio, motivado por objetivos educativos en lugar de por objetivos del proyecto.

36.2. Priorización de los riesgos del proyecto

Un método útil para priorizar los riesgos del proyecto global es estimar su probabilidad e impacto (en coste, tiempo o esfuerzo). La estimación podría ser cuantitativa (que normalmente son muy especulativas) o simplemente cualitativas (por ejemplo, alto-medio-bajo, basado en discusiones y votaciones con puntos en grupo). Los peores riesgos son, naturalmente, aquellos tanto probables como de impacto alto. Por ejemplo:

Riesgo	Probabilidad	Impacto	Ideas para atenuarlo
Número insuficiente y calidad de desarrolladores orientados a objetos expertos.	A	A	Leer el libro. Contratar consultores temporales Educación en aulas & tutorías Diseño y programación por parejas
Demostración no preparada para la próxima convención POS-World en Hamburgo.	M	A	Contratar consultores temporales que sean especialistas en el desarrollo de sistemas de PDV en Java. Identificar requisitos “atractivos” que queden bien en una demostración y darles prioridad, sobre los otros. Maximizar el uso de componentes prefabricados.
...	...	...	...

En términos de los artefactos del UP, esto forma parte del Plan de Desarrollo de Software.

36.3. Planificación adaptable vs. predictiva

Una de las grandes ideas del desarrollo iterativo es la adaptación basada en la retroalimentación, en lugar de intentar predecir y planificar *en detalle* el proyecto completo. En consecuencia, en el UP, uno crea un Plan de Iteración sólo para la *siguiente* iteración. Más allá de la siguiente iteración se deja abierto el plan detallado, para que se ajuste de manera adaptable en el futuro (ver Figura 36.1). Además de fomentar el comportamiento flexible y oportunista, una razón sencilla para no planificar el proyecto completo en detalle es que en el desarrollo iterativo no todos los requisitos, detalles de diseño y, por tanto, las etapas, se conocen al comienzo del proyecto¹. Otra es la preferencia de confiar en el juicio de planificación del equipo conforme ellos proceden. Finalmente, suponga que hubiese un plan detallado de grano fino al comienzo del proyecto, y el equipo se “desvía” de él por adquirir una nueva percepción de cómo ejecutar el proyecto de la mejor manera. Desde el exterior, esto podría verse como un tipo de fallo, cuando de hecho es justo lo contrario.

¹ Tampoco se conocen realmente o de manera fiable en un proyecto “en cascada”, aunque se planifique de manera detallada el proyecto completo como si se conocieran.

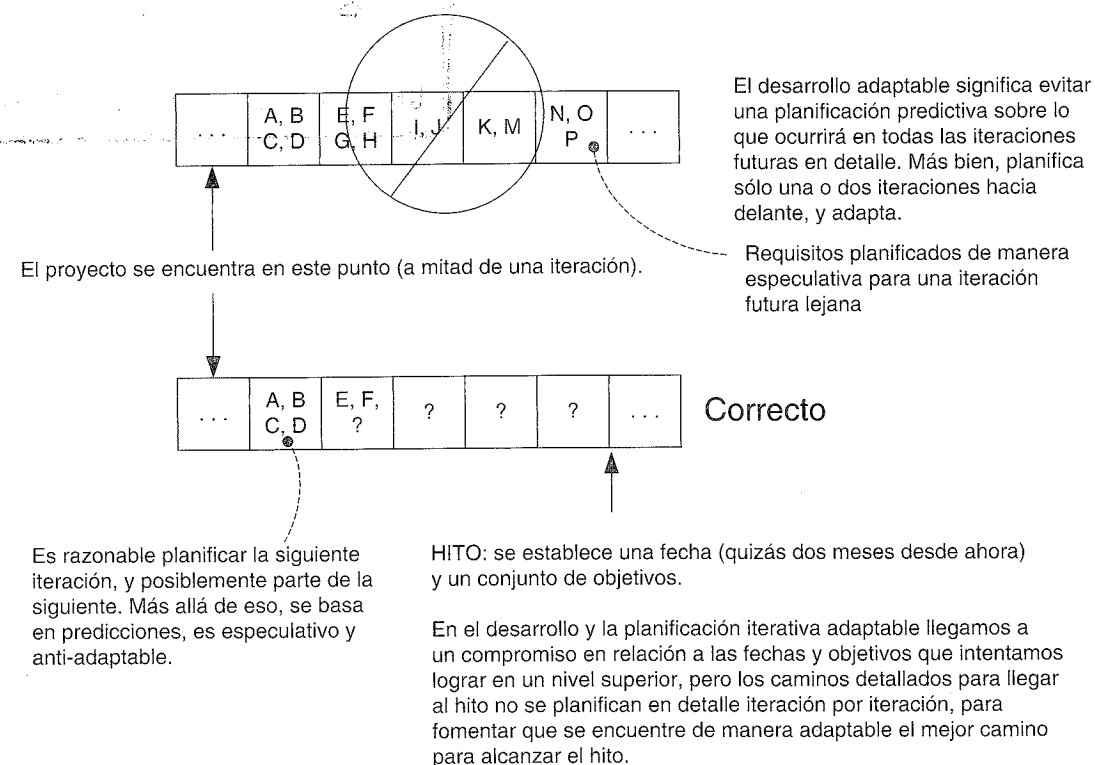


Figura 36.1. Los hitos son importantes, pero evite la planificación detallada que predice un futuro lejano.

Sin embargo, *existen* todavía objetivos e hitos; el desarrollo adaptable no significa que el equipo no sabe hacia dónde va, o las fechas de los hitos y objetivos. En el desarrollo iterativo, el equipo todavía llega a un compromiso en cuanto a las fechas y objetivos, pero el camino detallado para alcanzarlos es flexible. Por ejemplo, el equipo NuevaEra podría establecer un hito de que en tres meses se completarán los casos de uso *Procesar Venta*, *Gestionar Devoluciones* y *Autenticar Usuarios*, y las características de registrar y reglas conectables. Pero —y éste es el punto clave— no se define en detalle el plan o camino de grano fino de las iteraciones fijadas en dos semanas para ese hito. No se fija el orden de las etapas, o lo que se va a hacer en cada iteración a lo largo de los siguientes tres meses. Más bien, únicamente se planifica la siguiente iteración de dos semanas, y el equipo se adapta paso a paso, trabajando para cumplir los objetivos para la fecha de entrega. Por supuesto, las dependencias en componentes y recursos naturalmente restringen algo el orden del trabajo, pero no todas las actividades necesitan que se planifiquen en detalle de grano fino.

El personal involucrado externo ve un plan de macro-nivel (como al nivel de tres meses) al que el equipo se compromete. Pero la organización de micro-nivel se deja al mejor —y adaptable— juicio del equipo, cuando se beneficie de nuevas percepciones (ver Figura 36.1).

Finalmente, aunque en el UP se prefiere la planificación adaptable de grano fino, cada vez más es posible planificar con éxito hacia delante dos o tres iteraciones (con niveles crecientes de desconfianza) conforme los requisitos y la arquitectura se estabilizan, el equipo madura, y los datos se recogen a la velocidad del desarrollo.

36.4. Planes de Fase y de Iteración

A un macro-nivel, es posible establecer fechas y objetivos de los hitos; pero a micro-nivel, el plan para los hitos se deja flexible excepto para el futuro inmediato (por ejemplo, las siguientes cuatro semanas). Estos dos niveles se reflejan en el **Plan de Fase** y **Plan de Iteración** del UP, ambos forman parte del Plan de Desarrollo del Software compuesto. El Plan de Fase expone las fechas y objetivos de los hitos de macro-nivel, tales como los hitos del final de las fases y de las pruebas del sistema piloto a mitad de la fase. El Plan de Iteración define el trabajo para la iteración actual y la siguiente —no todas las iteraciones— (ver Figura 36.2).

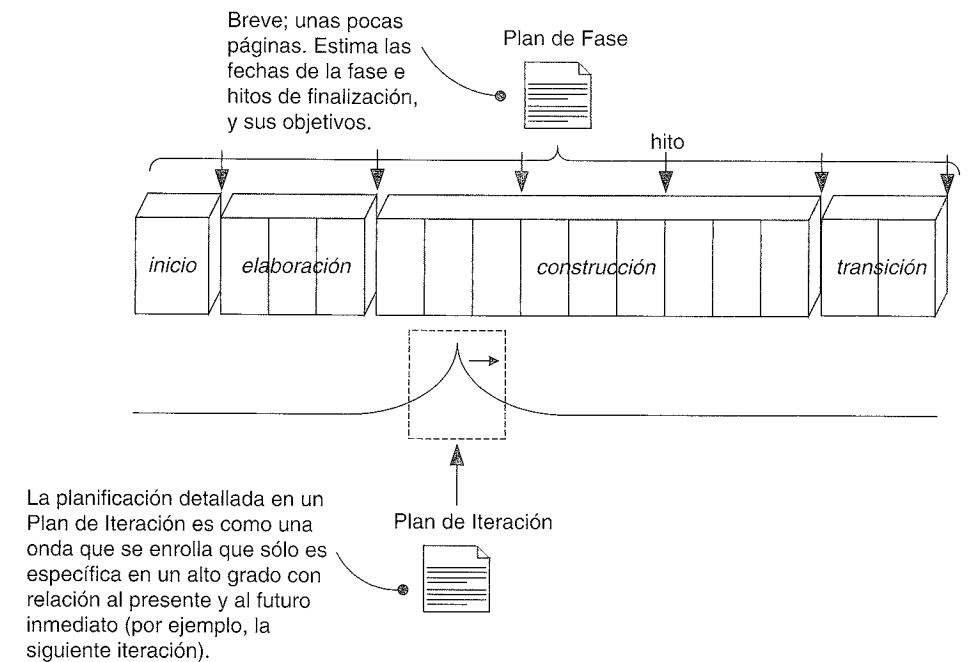


Figura 36.2. Planes de Fase y de Iteración.

Durante la fase de inicio, los hitos estimados en el Plan de Fase son vagos “estimados en base a conjeturas”. A medida que se progresa en la elaboración, las estimaciones se mejoran. Un objetivo de la fase de elaboración es tener, a su terminación, suficiente información realista para que el equipo llegue a un compromiso en relación con las fechas y objetivos de los hitos importantes para el final de la construcción y transición (esto es, la entrega del proyecto).

36.5. Plan de Iteración: ¿qué hacemos en la siguiente iteración?

El UP está dirigido por los casos de uso, lo que en parte implica que se organiza el trabajo alrededor de la finalización de los casos de uso. Es decir, se asigna una iteración para que implemente uno o más casos de uso, o escenarios de casos de uso cuando el

caso de uso completo sea demasiado complejo para una iteración. Y puesto que algunos requisitos no se expresan como casos de uso, sino como características, como registrar y reglas del negocio conectables, éstas también se deben asignar a una o más iteraciones (ver Figura 36.3).

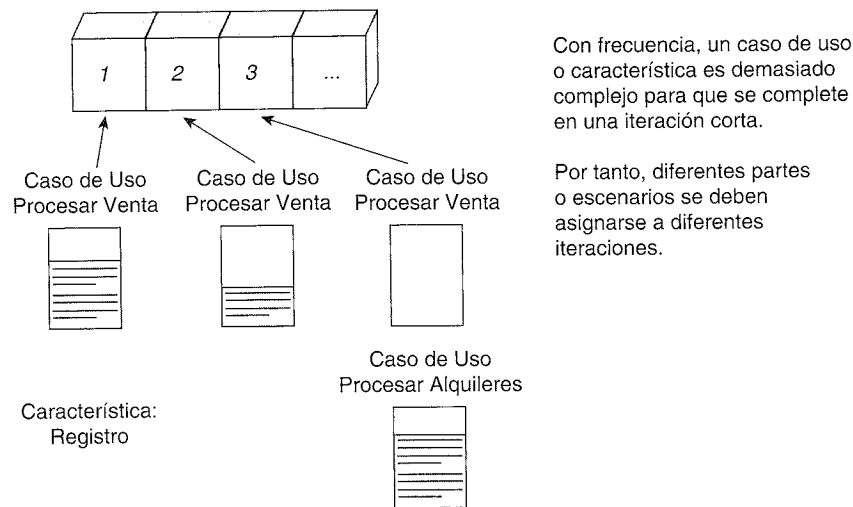


Figura 36.3. Trabajo asignado a una iteración.

Normalmente, se dedica la primera iteración de la elaboración a innumerables tareas generales como instalación y ajuste de herramientas y componentes, aclarar los requisitos, etcétera.

La priorización de los requisitos guía la elección del trabajo inicial. Por ejemplo, el caso de uso *Procesar Venta* es claramente importante. Por tanto, empezamos a abordarlo en la primera iteración. Aunque, no se implementan todos los escenarios de *Procesar Venta* en la primera iteración. Más bien, se escogen algunos escenarios simples, de camino feliz, como el de pago sólo en efectivo. Aunque el escenario es simple, su implementación comienza a desarrollar algunos elementos centrales del diseño.

Durante las iteraciones de la elaboración se abordarán requisitos diferentes, significativos para la arquitectura, relacionados con este caso de uso, forzando al equipo a tratar brevemente muchos aspectos de la arquitectura: las capas importantes, la base de datos, la interfaz de usuario, la interfaz entre los subsistemas importantes, etcétera. Esto nos lleva a la creación en las primeras etapas de una implementación “en anchura y superficial” a través de muchas partes del sistema —un objetivo común en la fase de elaboración—.

36.6. Traza de los requisitos a través de las iteraciones

La tarea de crear el primer Plan de Iteración nos trae una cuestión relevante en el desarrollo iterativo, que se ilustra en la Figura 36.3.

Como se indicó en la última sección, no se implementarán todos los escenarios de *Procesar Venta* en la primera iteración. De hecho, este complejo caso de uso podría tar-

dar en completarse muchas iteraciones de dos semanas a lo largo de un periodo de seis meses. Cada iteración abordará nuevos escenarios o partes de escenarios.

Cuando no es posible llevar a cabo todos los escenarios de un caso de uso en una iteración, surge un problema en la traza de los requisitos. ¿Cómo recoge uno qué partes del caso de uso se han completado, en cuáles se está trabajando actualmente, o todavía no se han hecho? Una herramienta de requisitos construida para este trabajo proporciona una solución.

Un ejemplo es la herramienta de Rational, RequisitePro, y merece la pena dedicar un momento a su estudio para entender cómo trabajan estas herramientas para mantener la traza de los casos de uso completados parcialmente a lo largo de las iteraciones. No es que propongamos esta herramienta, sino que se ofrece la presentación para ilustrar una solución a este importante problema de mantener la traza.

Un ejemplo de herramienta de gestión de requisitos

RequisitePro está integrada con Microsoft Word de manera que uno podría introducir y editar los requisitos en Word, seleccionar una frase y definir la frase seleccionada como un requisito al que se le va a seguir la pista en RequisitePro.

Cada requisito puede tener una variedad de atributos, como el estado, riesgo, etcétera (ver Figuras 36.4 y 36.5). Con una herramienta como ésta, se puede manejar el problema de mantener la traza de la terminación parcial de los casos de uso a lo largo de las iteraciones.

Todas las sentencias de los escenarios principales de éxito y las extensiones se pueden representar individualmente como requisitos a los que se les va a seguir la pista, y

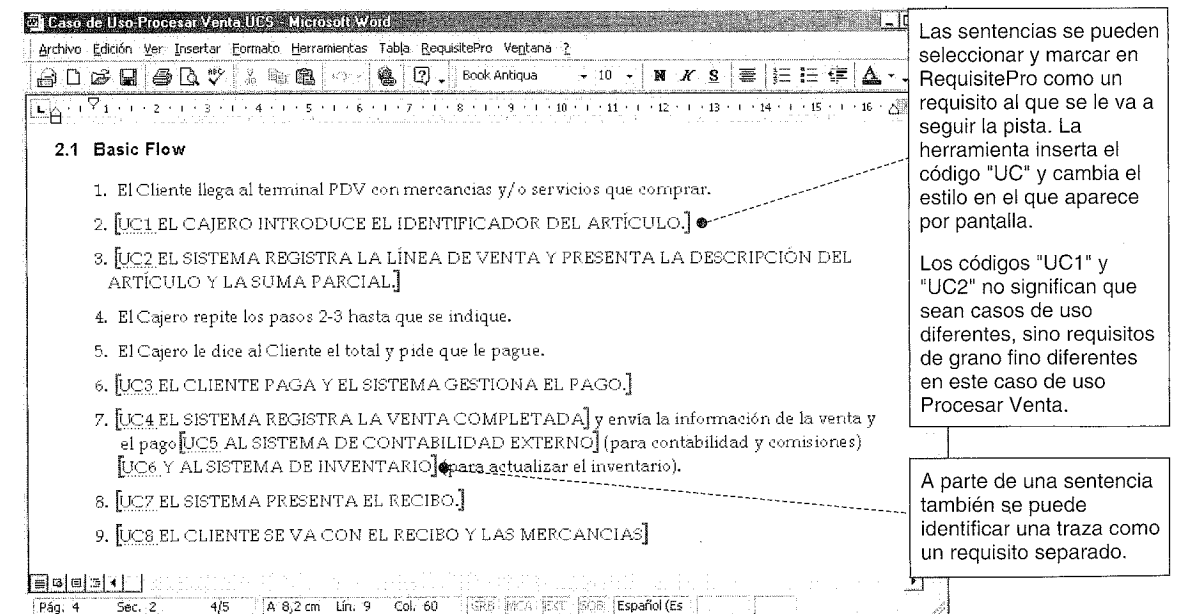


Figura 36.4. Etiquetado básico de frases de los casos de uso como requisitos.

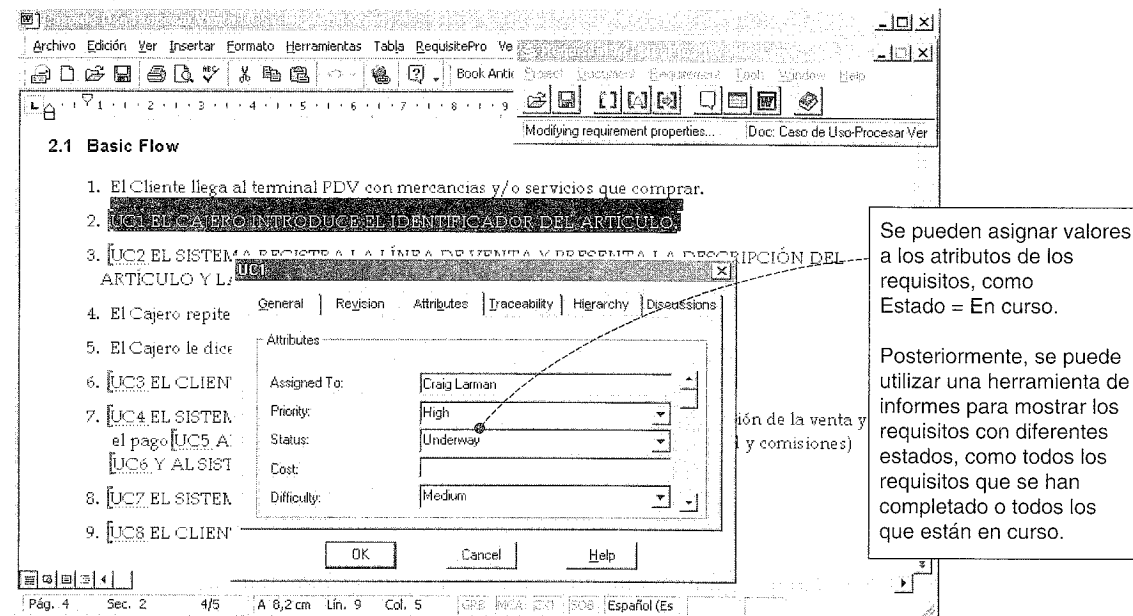


Figura 36.5. Cada requisito etiquetado tiene muchos atributos.

cada uno se puede identificar con varios valores de estado como *propuesto*, *aprobado*, etcétera.

36.7. La (in)validez de las primeras estimaciones

Basura entra, basura sale. Las estimaciones hechas con información poco fiable y difusa son poco fiables y difusas. En el UP se entiende que las estimaciones que se hacen durante la fase de inicio no son de fiar (esto se cumple para todos los métodos, pero el UP lo reconoce). Las estimaciones iniciales de la fase de inicio proporcionan una guía sobre si merece la pena que se haga un estudio real en la elaboración, para generar una buena estimación. Después de la primera iteración de la elaboración contamos con alguna información realista para producir una estimación aproximada. Después de la segunda iteración, la estimación comienza a cobrar credibilidad (ver Figura 36.6).

Las estimaciones útiles requieren de la investigación en algunas iteraciones de la elaboración.

Esto no significa que sea imposible o no merezca la pena intentar hacer estimaciones precisas al principio. Si es posible, muy bien. Sin embargo, la mayoría de las organizaciones no encuentran que éste sea el caso, por razones entre las que se encuentran la continua introducción de nuevas tecnologías, aplicaciones nuevas, y muchas otras complicaciones. Por tanto, el UP aboga por realizar un poco de trabajo realista en la elaboración antes de generar las estimaciones que se utilizan para la planificación del proyecto y la elaboración de los presupuestos.

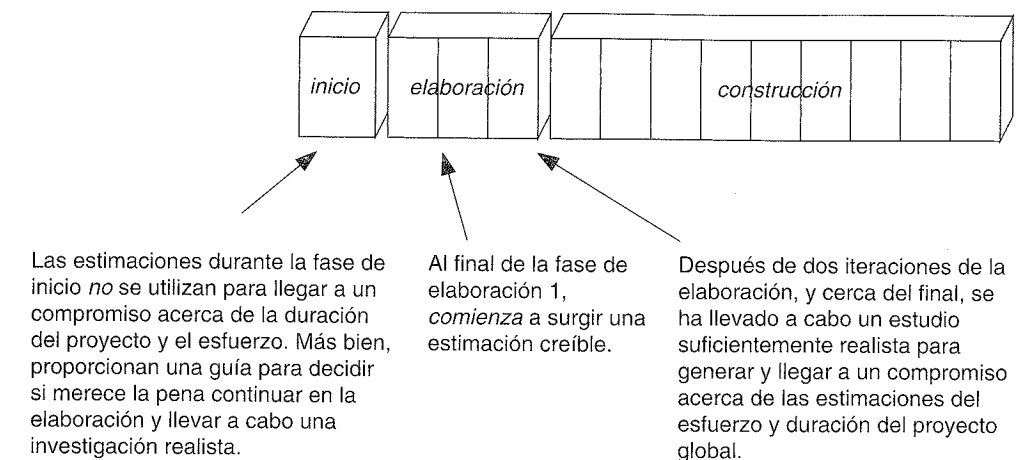


Figura 36.6. Estimación y fases del proyecto.

36.8. Organización de los artefactos del proyecto

El UP organiza los artefactos en función de las disciplinas. El Modelo de Casos de Uso y la Especificación Complementaria están en la disciplina de Requisitos. El Plan de Desarrollo del Software forma parte de la disciplina de Gestión del Proyecto, etcétera. Por tanto, organice carpetas en su control de versiones y sistema de directorios para reflejar las disciplinas, y coloque los artefactos de una disciplina en la carpeta de la disciplina relacionada (ver Figura 36.7).

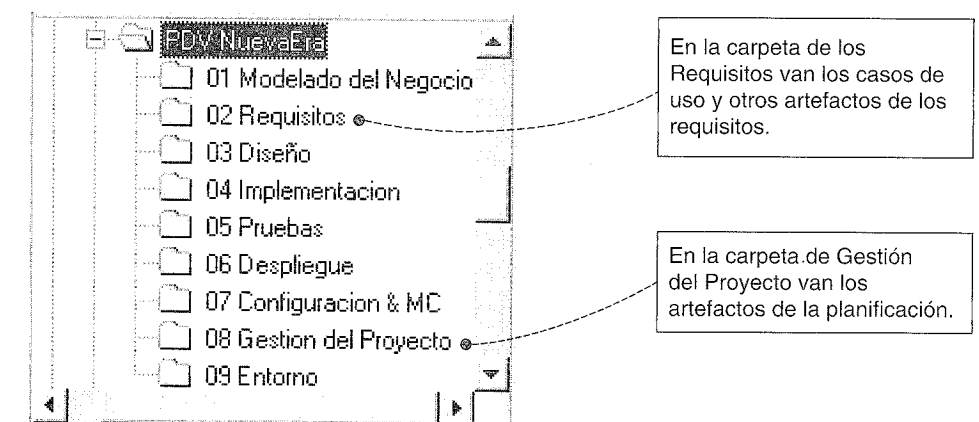


Figura 36.7. Organice los artefactos del UP en carpetas que se corresponden con sus disciplinas.

Esta organización funciona para la mayoría de los elementos que no son de la implementación. Algunos artefactos de la implementación, como la base de datos actual o los ficheros ejecutables, se encuentran comúnmente en diferentes ubicaciones debido a diversos motivos de la implementación.

Sugerencia

Después de cada iteración, utilice la herramienta de control de versiones para crear un punto de control etiquetado y congelado de todos los elementos en estas carpetas (incluyendo el código fuente). Habrá una versión "Elaboración-1", "Elaboración-2", y así sucesivamente, de cada artefacto. Para una estimación posterior de la velocidad del equipo (en éste o en otro proyecto), estos puntos de control proporcionan datos reales de la cantidad de trabajo que se hizo en cada iteración.

36.9. Algunas cuestiones de la planificación de la iteración del equipo

Equipos de desarrollo en paralelo

Un proyecto grande normalmente se divide en trabajos de desarrollo en paralelo, donde varios equipos trabajan en paralelo. Una manera de organizar los equipos es a lo largo de líneas de la arquitectura: por capas y subsistemas. Otra estructura organizacional es por conjuntos de características, que podrían muy bien corresponderse a la organización de la arquitectura.

Por ejemplo:

- Equipo de la capa del dominio (o equipo del subsistema del dominio)
- Equipo de la interfaz de usuario
- Equipo de internacionalización
- Equipo de servicios técnicos (equipo de persistencia, etc.)

Equipos en iteraciones de diferente longitud

Algunas veces, el desarrollo de un subsistema (como el servicio de persistencia) a cualquier nivel significativamente utilizable requiere mucho tiempo, especialmente durante sus primeras etapas. En lugar de alargar la longitud de la iteración global para todos los equipos, una alternativa es mantener la iteración corta (en general, un objetivo

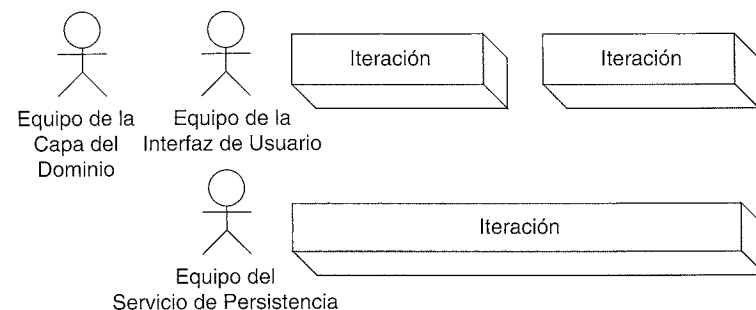


Figura 36.8. Diversas longitudes de las iteraciones.

respetable) para la mayoría de los equipos, y de longitud doble para el equipo "más lento" (ver Figura 36.8).

Velocidad del equipo y adopción incremental del proceso

Además de necesitar iteraciones más largas para los equipos muy grandes, otro motivo para alargar una iteración (por ejemplo, de tres semanas a cuatro), está relacionado con la velocidad y experiencia del equipo. Un equipo para el que son nuevas muchas de las prácticas y las tecnologías, naturalmente irá más despacio, o necesitará más tiempo para completar una iteración. Equipos con menos experiencia se beneficiarán de menos iteraciones *ligeramente* más largas que los equipos con más experiencia.

Nótese que el desarrollo iterativo proporciona un mecanismo para mejorar la estimación de la velocidad: el progreso real en las primeras iteraciones da a conocer las estimaciones para las iteraciones posteriores.

Relacionado con esto está la estrategia **adopción incremental del proceso**. En las primeras iteraciones, los equipos con menos experiencia asumen un conjunto pequeño de prácticas. A medida que los miembros del equipo los digieren y los dominan, añaden más —¡asumiendo que son útiles! Por ejemplo, en las primeras iteraciones el equipo podría construir y probar un sistema una vez al día. En iteraciones posteriores, podrían adoptar integraciones continuas y pruebas del sistema (que suceden muchas veces cada día) con una herramienta de integración continua como la de libre distribución Cruise Control (cruisecontrol.sourceforge.net).

36.10. No se entendió la planificación en el UP cuando...

- Todas las iteraciones se planifican de manera especulativa en detalle, prediciendo el trabajo y los objetivos de cada iteración.
- Se espera que las primeras estimaciones de la fase de inicio o de la primera iteración de la elaboración sean fiables, y se utilizan para llegar a compromisos del proyecto a largo plazo; generalizando, se esperan estimaciones fiables con investigaciones triviales o ligeras.
- Los problemas sencillos o las cuestiones de bajo riesgo se abordan en las primeras iteraciones.

Si la estimación de una organización y el proceso de planificación se parece algo al que se presenta a continuación, no se entendió la planificación que propone el UP:

1. Al comienzo de una fase de planificación anual, se identifican nuevos sistemas o características a un alto nivel; por ejemplo, "Sistemas Web para la gestión de la contabilidad".
2. A los encargados técnicos se les da poco tiempo para estimar de manera especulativa el esfuerzo y la duración de grandes proyectos, caros o arriesgados, que involucran con frecuencia nuevas tecnologías.

- 3. Se establece la planificación y presupuesto de los proyectos para el año.
- 4. Las personas involucradas se preocupan cuando los proyectos reales no se corresponden con las estimaciones originales. Ir al Paso 1.

Este enfoque carece de una estimación realista y refinada iterativamente basada en investigaciones serias como promueve el UP.

36.11. Lecturas adicionales

Software Project Management: A Unified Framework de Royce proporciona una perspectiva iterativa y del UP sobre la gestión y planificación de proyectos.

Surviving Object-Oriented Projects: A Manager's Guide de Cockburn contiene más información útil acerca de la planificación iterativa, y la transición a los proyectos iterativos que hacen uso de la tecnología de objetos.

The Rational Unified Process: An Introduction de Kruchten contiene capítulos útiles dedicados especialmente a la gestión y planificación de proyectos en el UP.

Una advertencia, hay algunos libros que se plantean discutir la planificación para el “desarrollo iterativo” o el “Proceso Unificado” que realmente esconden un enfoque de planificación en cascada o predictivo.

Rapid Development [McConnell96] ofrece una excelente visión global de muchas prácticas y cuestiones relacionadas con la gestión y planificación del proyecto, y riesgos del proyecto.

COMENTARIOS ACERCA
DEL DESARROLLO ITERATIVO
Y EL UP

Deberías utilizar el desarrollo iterativo sólo en los proyectos en los que quieras tener éxito.

Martin Fowler

Objetivos

- Introducir y ampliar algunos temas del UP.
- Introducir otras prácticas aplicables al desarrollo iterativo.
- Estudiar cómo el ciclo de vida iterativo puede ayudar a reducir algunos problemas del desarrollo.

37.1. Buenas prácticas y conceptos del UP adicionales

La idea más importante que hay que apreciar y poner en práctica en el UP es el desarrollo adaptable, con iteraciones cortas con una duración fija. Entre las buenas prácticas y conceptos claves adicionales del UP encontramos:

- **Abordar las cuestiones de alto riesgo y valor en las primeras iteraciones:** Por ejemplo, si el nuevo sistema es un servidor de aplicaciones que tiene que gestionar 2.000 clientes concurrentes con tiempo de respuesta de la transacción por debajo del segundo, no espere muchos meses (o años) para diseñar e implementar este requisito de alto riesgo. Más bien, céntrese rápidamente en diseñar, programar y probar los componentes esenciales del software y la arquitectura para esta cuestión arriesgada; deje el trabajo fácil hasta las iteraciones posteriores. La idea es disminuir los riesgos altos en las primeras iteraciones, de manera que el proyecto no “fracase tarde”, que es una característica de los proyectos en cascada que retrasan los temas difíciles y arriesgados hasta etapas posteriores del ciclo de vida. Es mejor “fracasar pronto”, si es que fracasa, haciendo primero las cosas difíciles. De

este modo, se dice que el UP está **dirigido por el riesgo**. Finalmente, obsérvese que el riesgo llega de muchas formas: falta de habilidades o recursos, desafíos técnicos, facilidad de uso, política, etcétera. Todas estas formas influyen en lo que se va a abordar en las primeras iteraciones.

- **Usuarios involucrados continuamente:** El desarrollo iterativo y el UP tratan de dar pequeños pasos rápidamente y obtener retroalimentación. Esto requiere una continua atención e implicación por parte de las personas involucradas del negocio y los expertos en la materia de estudio para clarificar y dirigir el proyecto. Al principio, el personal del negocio podría sentir que esto es una imposición. Sin embargo, existe una correlación entre la mayoría de los proyectos que han fracasado y la falta de implicación de los usuarios [Standish94], y este enfoque le proporciona al personal del negocio la capacidad de dar forma al software como realmente lo necesitan. En proyectos donde el “usuario” puede ser cualquiera, como un nuevo sitio web o producto del consumidor, grupos interesados podrían actuar como sustitutos.
- **Atención en las primeras etapas a construir una arquitectura básica cohesiva:** Es decir, el UP está **centrado en la arquitectura**. Esto está relacionado con abordar las cuestiones de alto riesgo en las primeras iteraciones, puesto que normalmente establecer los elementos básicos de la arquitectura es un elemento arriesgado o crítico. Las primeras iteraciones se centran en una implementación de la arquitectura “en anchura y superficial”, estableciendo las cuestiones de diseño importantes, y los subsistemas con sus interfaces y responsabilidades. El equipo “investigará” en áreas verticalmente profundas sobre requisitos específicos difíciles o arriesgados, como el requisito para transacciones por debajo del segundo con 2.000 clientes concurrentes.
- **Verificar continuamente la calidad, desde el principio y con frecuencia:** La calidad en este contexto abarca satisfacer o superar correctamente los requisitos en un proceso sostenible y repetible, con software de fácil mantenimiento y escalable. Una razón para una campaña temprana, continua e intensiva de pruebas, inspección y aseguramiento de la calidad es que el coste de un defecto tardío se incrementa de manera no lineal a través de las fases de un proyecto. Además, el desarrollo iterativo se basa en la retroalimentación y la adaptación; por tanto, las pruebas y evaluación realistas en las primeras etapas son actividades críticas para obtener retroalimentación significativa. Esto difiere del proyecto en cascada, donde la etapa de aseguramiento de la calidad se hace cerca del final del proyecto, cuando la respuesta es la más difícil y costosa. En el UP, la verificación de la calidad se integra de manera continua desde el principio, de manera que no hay grandes sorpresas cerca del final del proyecto. Observe que en el UP, la verificación de la calidad también se refiere a la calidad del proceso —cada iteración, evaluando lo bien que lo está haciendo el equipo—.
- **Aplicar casos de uso:** Informalmente, los casos de uso son historias escritas del uso de un sistema. Son mecanismos para explorar y recoger los requisitos funcionales, a diferencia del estilo más antiguo de la lista de funciones o la lista “El sistema deberá hacer...”. El UP recomienda que se apliquen los casos de uso como la forma principal para la captura de los requisitos, y como una fuerza que dirige la planificación, diseño, pruebas y la escritura de la documentación del usuario final.

- **Modelar el software visualmente:** Un porcentaje muy importante del cerebro de las personas está implicado en el procesamiento visual, que es una de las motivaciones que hay detrás de la presentación visual o gráfica de la información [Tufte92]. Por tanto, es conveniente emplear no sólo lenguajes textuales (como texto o código), sino también lenguajes simbólicos, basados en diagramas, visuales con orientación espacial, como UML, porque esto se beneficia de las ventajas naturales del cerebro¹. Además, la *abstracción* es una práctica útil para razonar sobre los diseños del software y comunicarlos, porque esto nos permite centrarnos en los aspectos importantes, mientras ocultamos o ignoramos los detalles que producen ruido. Un lenguaje visual como UML nos permite visualizar y razonar sobre los modelos abstractos del software, pasando rápidamente al diseño con esquemas en forma de diagramas de las grandes ideas. Pero como se estudiará más tarde, existe un “punto dulce en UML” entre realizar pocos o demasiados diagramas.
- **Gestión de requisitos cuidadosa:** Esto no significa hacer uso del estilo en cascada de definir completamente y congelar los requisitos en la primera fase del proyecto. Más bien, implica no ser descuidado; es decir, ser habilidoso en la elicitación, escritura, asignación de las prioridades, establecimiento de las trazas y mantenimiento de la pista a lo largo del ciclo de vida de los requisitos, normalmente con el soporte de una herramienta. Esto parece obvio, pero da la impresión de que rara vez se pone en práctica bien. La gestión de requisitos pobre es un factor común en proyectos que tienen problemas [Standish94].
- **Control de cambios:** Esta práctica comprende varias ideas: Primero, gestión de peticiones de cambios. Aunque un proyecto UP iterativo acepta los cambios, no acepta el caos. Cuando surge la solicitud de un nuevo requisito durante las iteraciones, en lugar de un jovial “¡Por supuesto, sin problemas!” hay una evaluación racional del esfuerzo y el impacto, y si se acepta, se modifica la planificación. También incluye la idea de mantener la pista del ciclo de vida de todas las peticiones de cambio (solicitado, en curso,...). Segundo, gestión de configuraciones. Se utilizan herramientas de gestión de configuraciones y construcción para dar soporte a la frecuente (idealmente, por lo menos diariamente) integración y pruebas del sistema, desarrollo en paralelo, áreas de trabajo de desarrolladores y configuraciones separadas, y control de versiones —desde el comienzo del proyecto—. En el UP, todos los bienes del proyecto (no sólo el código) deberían estar bajo el control de configuraciones y versiones.

37.2. Las fases de construcción y transición

Construcción

La elaboración termina cuando se han resuelto las cuestiones de alto riesgo, se ha completado el núcleo central o esqueleto de la arquitectura y se han entendido “la mayoría” de los requisitos. Al final de la elaboración, es posible estimar de manera más realista el esfuerzo y duración restante del proyecto.

¹ Éste también es un motivo para utilizar colores en los diagramas (a no ser que algún miembro del equipo sea daltónico). Por ejemplo, véase [CDL99].

Le sigue la **fase de construcción**, cuyo propósito es esencialmente terminar de construir la aplicación, realizar pruebas alpha, preparar las pruebas beta (en la fase de transición) y preparar el despliegue, mediante actividades tales como la escritura de las guías de usuario y la ayuda on-line. A veces, se resume como poner la “carne sobre el esqueleto” creado en la elaboración. Mientras que la elaboración se puede caracterizar como que construye el núcleo del sistema arriesgado y significativo para la arquitectura, la construcción se puede describir como que construye el resto. Como antes, el desarrollo avanza por medio de una serie de iteraciones en las que se fija la duración. En cuanto al personal, se recomienda que se utilice un equipo pequeño y cohesivo durante la elaboración, y luego ampliar el tamaño del equipo durante la construcción; además probablemente habrá más equipos en paralelo desarrollando durante esta fase.

Transición

La construcción termina cuando se considera que el sistema está preparado para el despliegue operacional, y se han completado todos los materiales de soporte, como las guías de usuario, materiales de aprendizaje, etcétera. Le sigue la **fase de transición**, cuyo propósito es poner el sistema en producción. Esto podría incluir actividades como pruebas beta, reaccionar a la retroalimentación de las pruebas beta, pequeños ajustes, conversión de datos, cursos de entrenamiento, marketing para el lanzamiento del producto, funcionamiento en paralelo del antiguo y el nuevo sistema, y cosas por el estilo.

37.3. Otras prácticas interesantes

Ésta no es una lista exhaustiva, pero entre las prácticas interesantes —no documentadas explícitamente en el UP— que han sido de valor en los proyectos iterativos se encuentran:

- El patrón de proceso **SCRUM** [BDSS00]; véase también www.controlchaos.com. El más concreto es una reunión de SCRUM diaria de pie de “15 minutos”. El jefe del proyecto pregunta a cada persona: 1) las cosas hechas desde la última reunión; 2) los objetivos para el día siguiente; y 3) los obstáculos para que el jefe los elimine. Yo también he preguntado a cada miembro las percepciones relevantes que quiere compartir con el equipo. La reunión fomenta el comportamiento del equipo adaptable que emerge, medidas del progreso de grano fino, comunicación de alta densidad y la socialización de los proyectos. Otras ideas claves son: el equipo está libre de todas las distracciones externas, no se le añade ningún trabajo adicional (desde fuera del equipo) durante una iteración, y el trabajo de gestión es eliminar todos los obstáculos y distracciones, de manera que el equipo pueda centrarse.
- Algunas prácticas de la **Programación Extrema** (XP, *Extreme Programming*) [Beck00], como **programar probando primero**: escribir una prueba de unidad *antes* del código que se va a probar, y escribir las pruebas para casi todas las clases. Si estamos trabajando con Java, **JUnit** (www.junit.org) es un framework de pruebas conocido de libre de distribución. Escriba un poco de las pruebas, escriba un poco de código, haga que pase las pruebas, repita. Es esencial que se escriban las pruebas *en primer lugar* para experimentar el valor de este enfoque.

- **Integración continua**, otra práctica de XP; véase [Beck00] para obtener una introducción y www.martinfowler.com para los detalles. El UP incluye la buena práctica de integrar el sistema completo al menos una vez en cada iteración. Esto se abrevia con frecuencia como la práctica de la **construcción diaria**. Integración *continua* lo acorta todavía más, integrando todo el nuevo código que se registra (al menos) cada pocas horas. Aunque esto se puede hacer manualmente, una alternativa es utilizar un entorno que automatice la integración continua y las pruebas en una máquina para la construcción rápida que ejecuta un proceso demonio. Periódicamente se despierta (como cada dos minutos) y busca nuevo código para registrar, que dispara la ejecución de un *script* de prueba y reconstrucción. Está disponible libremente un sistema de integración continua para los proyectos Java denominado **Cruise Control** de libre distribución en SourceForge (cruisecontrol.sourceforge.net).

37.4. Motivos para fijar la duración de una iteración

Hay por lo menos cuatro motivos para fijar la duración de una iteración.

Primero, la **ley de Parkinson**. Parkinson con decepción observó que “El trabajo se expande de manera que rellena el tiempo disponible para su terminación” [Parkinson58]. Fechas de finalización distantes o difusas (por ejemplo, dentro de seis meses), agravan este efecto. Cerca del comienzo del proyecto, puede parecer que hay mucho tiempo para proceder sin prisa. Pero si la fecha de finalización para la siguiente iteración es sólo dentro de *dos semanas*, y se debe colocar un sistema parcial ejecutable y probado para esta fecha, el equipo tiene que centrarse, tomar decisiones y moverse.

Segundo, **asignar prioridades y decisión**. Las iteraciones cortas con la duración fija fuerzan al equipo de desarrollo a tomar decisiones teniendo en cuenta la prioridad del trabajo y los riesgos, identificar cuáles tienen el valor más alto para el negocio o técnico, y estimar algo de trabajo. Por ejemplo, si está embarcado en la primera iteración, que se ha elegido que dure exactamente cuatro semanas, no hay mucha libertad para ser vago —se deben tomar decisiones concretas sobre lo que se hará realmente en las cuatro primeras semanas—.

Tercero, **satisfacción del equipo**. Las iteraciones cortas con la duración fija nos llevan a una rápida y repetida sensación de terminación, competencia y finalización. En ciclos ordinarios de dos o cuatro semanas, el equipo tiene la experiencia de terminar algo, en lugar de trabajar lentamente durante meses sin terminar nada. Estos factores psicológicos son importantes para la satisfacción del trabajo individual, y para generar confianza en el equipo.

Cuarto, **confianza del personal involucrado**. Cuando un equipo llega a un compromiso público de producir algo ejecutable y estable en un periodo corto de tiempo, en una fecha concreta, como dentro de dos semanas, y lo hace, el personal del negocio y otras personas implicadas incrementan su confianza en el equipo y en el proyecto.

37.5. El ciclo de vida secuencial en “cascada”

A diferencia del ciclo de vida iterativo del UP, una antigua alternativa es el ciclo de vida secuencial, lineal, o “en cascada” [Royce70], asociado a procesos grandes y predictivos.

En la práctica habitual, un ciclo de vida en cascada define etapas parecidas a las siguientes:

1. Clarificar, recoger y llegar a un compromiso de un conjunto de requisitos finales.
2. Diseñar un sistema en base a estos requisitos.
3. Implementar, basado en el diseño.
4. Integrar módulos dispares.
5. Evaluar y probar para conseguir corrección y calidad.

Un proceso de desarrollo basado en el ciclo de vida en cascada se asocia con estos comportamientos o aptitudes:

- Definición completa y cuidadosa de un artefacto (por ejemplo, los requisitos o el diseño) antes de pasar a la siguiente etapa.
- Comprometerse a un conjunto congelado de requisitos detallados.
- La desviación de los requisitos o el diseño durante las etapas posteriores indican un fallo porque no se ha sido lo suficientemente habilidoso o exhaustivo. ¡La próxima vez, intente con más empeño hacerlo bien!

Un proceso en cascada es similar al enfoque de la ingeniería según el cual se construyen los edificios o los puentes. Su adopción hace que el desarrollo del software parezca más estructurado y análogo a la ingeniería en algún otro campo. Durante algún tiempo, el proceso en cascada fue el enfoque que aprendieron más desarrolladores de software, directores, autores y profesores cuando eran estudiantes (y después repetían), sin investigar de manera crítica su aplicabilidad al desarrollo del software.

Algunas cosas se deberían construir como los edificios —tales como, bien... edificios— pero, normalmente, no el software.

Como se mencionó en el capítulo preliminar acerca del UP, el estudio de dos años presentado en el *MIT Sloan Management Review*, acerca de los proyectos software exitosos identificó cuatro factores comunes para el éxito: el desarrollo iterativo, en lugar del ciclo de vida en cascada, era el primero de la lista [MacCormack01].

Algunos problemas con el ciclo de vida en cascada

La metáfora de la construcción ha sobrevivido a su utilidad. Es el momento de cambiar de nuevo. Si, como creo, las estructuras conceptuales que construimos hoy son demasiado complicadas para que se especifiquen con precisión por adelantado, y demasiado complejas para que se construyan sin errores, entonces debemos utilizar un enfoque radicalmente diferente (desarrollo iterativo, incremental).

—Frederick Brooks, “No Silver Bullet”, *The Mythical Man-Month*

En una cierta escala de tiempo, realizar parte de los requisitos antes del diseño, y parte del diseño antes de la implementación, es inevitable y sensato. Para un proyecto corto de dos meses, se puede utilizar un ciclo de vida secuencial. Y una iteración individual en el desarrollo iterativo es como un proyecto en cascada corto.

Sin embargo, comienzan a aparecer las dificultades cuando la escala de tiempo se alarga. La complejidad pasa a ser alta, las decisiones especulativas se incrementan y complican, no existe retroalimentación, y en general las cuestiones de alto riesgo no se están abordando lo suficientemente pronto. Por definición, uno intenta todo o la mayoría del trabajo de los requisitos para el sistema completo antes de avanzar, y la mayoría del diseño antes de pasar a lo siguiente.

Se abordan etapas largas en las que se toman muchas decisiones sin el beneficio de una retroalimentación concreta a partir de implementaciones y pruebas reales. En la escala de un mini-proyecto de dos semanas (es decir, una iteración), se puede utilizar una secuencia lineal requisitos-diseño-implementación; el grado de compromisos especulativos de algunos requisitos y del diseño no se encuentra en la zona peligrosa. Sin embargo, a medida que se amplía la escala, así lo hace la especulación y el riesgo.

Entre los problemas relacionados con un proceso en cascada a la escala del proyecto completo encontramos:

- Mitigación del riesgo atrasada; abordando tarde problemas de alto riesgo o difíciles.
- Especulación e inflexibilidad de los requisitos y el diseño
- Alta complejidad
- Baja adaptabilidad

Mitigación de algunos problemas con el ciclo de vida iterativo

El desarrollo iterativo no es una bala mágica para el desafío del desarrollo del software. Pero, ayuda a reducir algunos problemas agravados por un ciclo de vida en cascada lineal.

Problema: Mitigación del riesgo atrasada

El riesgo aparece de muchas formas: el diseño incorrecto, el conjunto de requisitos equivocado, un entorno político extraño, falta de habilidades o recursos, facilidad de uso, etcétera.

En un ciclo de vida en cascada, *no* existe un intento activo de identificar y mitigar en primer lugar las cuestiones de más riesgo. Por ejemplo, la arquitectura incorrecta para un sitio web de alta carga y alta disponibilidad puede causar retrasos costosos, o cosas peores. En un proceso en cascada, la validación de la adecuación de la arquitectura tiene lugar *mucho tiempo* después de que se especifiquen (inevitablemente de manera imperfecta) todos los requisitos y todo el diseño, durante la importante etapa posterior de la implementación. Esto podría ser muchos meses e incluso años después de la fase de inicio del proyecto (ver Figura 37.1). Y no hay escasez de historias donde equipos separados hayan construido subsistemas a lo largo de un largo periodo, y entonces intentaran integrar éstos y comenzar las pruebas del sistema global cerca del final del proyecto —previsiblemente con resultados dolorosos—.

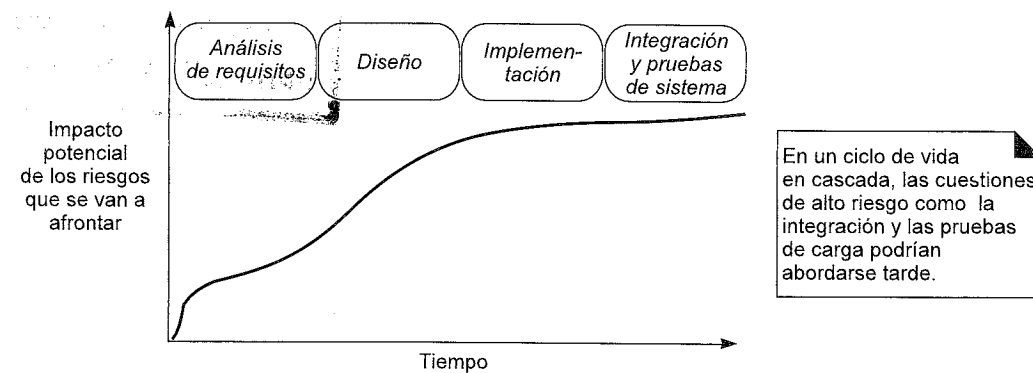


Figura 37.1. Ciclo de vida en cascada y los riesgos.

Mitigación

En cambio, en el desarrollo iterativo el objetivo es identificar y mitigar las cuestiones de más riesgo cuanto antes. Los riesgos altos podrían encontrarse en el diseño del núcleo de la arquitectura, la facilidad de uso de las interfaces, personal involucrado que no se implica. Sea lo que sea, se abordan en primer lugar. Como se ilustra en la Figura 37.2, las primeras iteraciones se centran en disminuir el riesgo. Continuando con el ejemplo anterior del sitio web con carga alta, en un enfoque iterativo, antes de realizar mucho estudio en otros requisitos o trabajo de diseño, el equipo, en primer lugar, diseña, implementa y prueba de manera realista lo suficiente del núcleo de la arquitectura para demostrar que se encuentran en el camino adecuado con respecto a la carga y a la disponibilidad. Si las pruebas demuestran que están equivocados, adaptan el diseño básico en las primeras etapas del proyecto, en lugar de cerca del final.

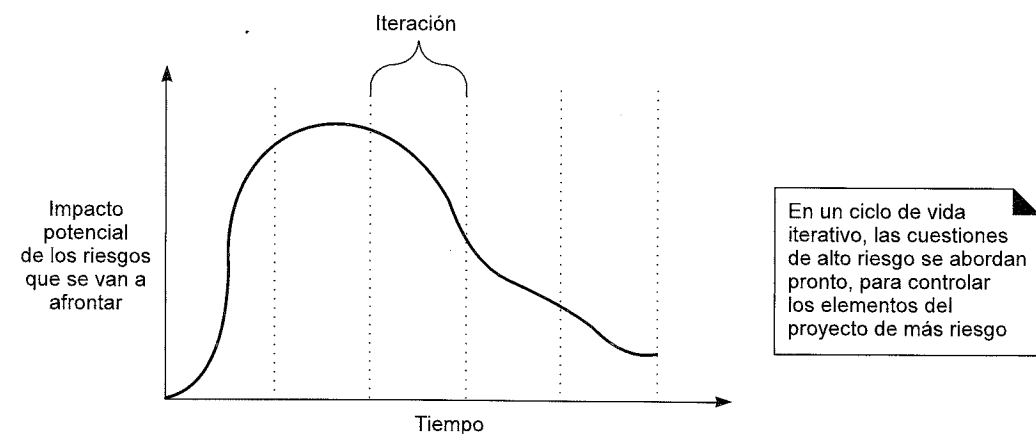


Figura 37.2. Ciclo de vida iterativo y los riesgos.

Problema: Especulación e inflexibilidad de los requisitos

Una suposición fundamental en un proceso en cascada es que los requisitos se pueden especificar por completo y después se congelan en la primera fase de un proyecto. En ta-

les proyectos, se hace un esfuerzo por realizar en primer lugar el análisis de requisitos completo, culminando en un conjunto de artefactos de requisitos que se revisan y se "dan por concluidos".

Normalmente resulta ser una suposición que tiene algún fallo. El esfuerzo para obtener todos los requisitos definidos y dados por concluidos antes del trabajo de diseño e implementación es probable, irónicamente, que incremente las dificultades del proyecto en lugar de mejorarlas. También lo hace difícil responder más tarde en un proyecto a una nueva oportunidad del negocio por medio de un cambio en el software.

Garantizado, hay algunos proyectos que necesitan que se realice en primer lugar un esfuerzo para especificar completamente y con precisión los requisitos. Esto es especialmente cierto cuando el software se acopla con la construcción de componentes físicos. Entre los ejemplos encontramos los dispositivos de aviación y médicos. Pero nótese que incluso en este caso, el desarrollo iterativo puede todavía aplicarse con provecho al proceso de diseño e implementación.

La investigación que suscita mayor interés en la crítica del mito de ser capaz de definir en primer lugar, con éxito, todos los requisitos procede de [Jones97]. Como se ilustra en la Figura 37.3, en este amplio estudio de 6.700 proyectos, los requisitos arrastrados —aquellos que no se anticiparon cerca del comienzo— son un hecho significativo de la vida del desarrollo del software, variando desde alrededor del 25% en proyectos de tamaño medio, hasta el 50% en los más amplios; Boehm y Papaccio presentan conclusiones basadas en investigaciones análogas en [BP88]. Las actitudes en cascada, que luchan en contra de (o simplemente niegan) este hecho, asumiendo que los requisitos y los diseños se pueden especificar y congelar, son incongruentes con la mayoría de las realidades de los proyectos.

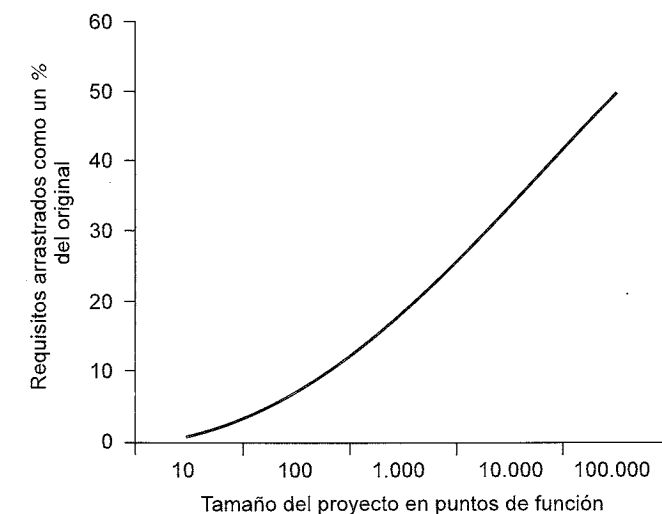


Figura 37.3. Los requisitos que cambian son una fuerza inevitable del desarrollo².

² Los puntos de función describen la complejidad del sistema con una métrica independiente del lenguaje de programación (véase www.ifpug.org).

Por tanto, “la única constante es el cambio”, normalmente porque:

- Las personas involucradas cambian de idea o no pueden imaginarse lo que quieren hasta que no ven un sistema concreto³.
- El mercado cambia.
- La especificación validada correctamente, detallada, y precisa es un desafío psicológico y organizativo para la mayoría de las personas involucradas [Kruchten00].

Y así, existen problemas previsibles y que se ve con frecuencia que surgen en los proyectos en cascada. Puesto que en realidad el cambio significativo es inevitable, éstos incluyen:

- Como se describió anteriormente, descubrimiento y mitigación de altos riesgos retrasado.
- Un sentimiento negativo entre los miembros del equipo de “vivir una ficción” o fracaso del proyecto, ya que la realidad de los cambios no se corresponde con el ideal.
- La realización de una amplia (y costosa) inversión en el diseño e implementación incorrectos (puesto que se basa en requisitos incorrectos).
- Falta de sensibilidad ante los deseos de los usuarios, o las oportunidades de mercado, que cambian.

Mitigación

En el desarrollo iterativo, no se especifican todos los requisitos antes del diseño y la implementación, y no se estabilizan los requisitos hasta después de, al menos, varias iteraciones. Por ejemplo:

Primero, se define un subconjunto de los requisitos básicos, por ejemplo, en un taller de requisitos de dos días. Entonces, el equipo selecciona un subconjunto de éstos para el diseño y la implementación (normalmente en base al riesgo o valor de negocio más alto). Después de una iteración de cuatro semanas, el personal involucrado se reúne en un segundo taller de requisitos de uno o dos días, revisan de manera intensiva el sistema parcial y aclaran y modifican sus solicitudes. Tras una segunda iteración (más corta) de dos semanas en la que se implementa incrementalmente el sistema, las personas involucradas se reúnen en un tercer taller de requisitos, y refinan de nuevo. A estas alturas, los requisitos comienzan a estabilizarse y representar el verdadero alcance y las intenciones clarificadas de las personas involucradas. A estas alturas, es posible determinar un plan algo realista y una estimación del trabajo restante. Estas iteraciones podrían caracterizarse como parte de la fase de elaboración del UP.

Todavía se admiten cambios posteriores en los requisitos. Sin embargo, la interacción en las primeras iteraciones del trabajo de implementación paralelo y el análisis de requisitos que obtiene retroalimentación a partir de la implementación parcial, nos lleva a una mejor definición de los requisitos en la fase de elaboración.

³ Barry Boehm ha llamado a esto el efecto “Lo sabré cuando lo vea”.

Problema: Especulación e inflexibilidad del diseño

Otra idea central del ciclo de vida en cascada es que la arquitectura y la mayor parte del diseño pueden y deberían especificarse por completo en la segunda fase importante de un proyecto, una vez que se han aclarado los requisitos. En tales proyectos, se hace un esfuerzo por describir a fondo la arquitectura completa, los diseños de los objetos, la interfaz de usuario, el esquema de base de datos, etcétera, antes del que comience la implementación. Algunos problemas asociados con esta suposición:

1. Puesto que los requisitos cambiarán, el diseño original podría no ser fiable.
2. Herramientas, componentes y entornos inmaduros o mal entendidos, hacen las decisiones de diseño especulativas y arriesgadas; podría demostrarse que son incorrectas sobre la implementación porque “no se suponía que el servidor de aplicación fuera a hacer eso...”.
3. En general, la falta de retroalimentación para probar o desaprobar el diseño, hasta mucho tiempo después de que se tomaran las decisiones de diseño.

Mitigación

Estos problemas se mitigan en el desarrollo iterativo construyendo rápidamente parte del sistema y validando el diseño y los componentes de terceras partes mediante las pruebas.

37.6. Ingeniería de usabilidad y diseño de interfaces de usuario

No existe probablemente ninguna técnica con mayor disparidad entre su importancia para el éxito en el desarrollo de software y la falta de una atención y educación formal que la **ingeniería de usabilidad** y el diseño de las interfaces de usuario (UI). Aunque fuera del alcance de esta introducción al A/DOO y el UP, nótese que el UP no incluye el reconocimiento de esta actividad; modelos de usabilidad y UI forman parte de la disciplina de Requisitos. En terminología del UP, los **guiones de los casos de uso** se pueden utilizar para describir de manera abstracta los elementos de la interfaz, y la navegación entre ellos, ya que se relacionan con los escenarios de los casos de uso.

Entre los libros útiles encontramos *Software for Use* de Constantine y Lockwood, *The Usability Engineering Lifecycle* de Mayhew, y *GUI Bloopers* de Johnson.

37.7. El Modelo de Análisis del UP

El UP contiene un artefacto denominado **Modelo de Análisis**; no es necesario y pocos lo crean. Quizás al **Modelo de Análisis** no se le ha asignado el nombre ideal, ya que es realmente un tipo de modelo de diseño. En el uso convencional (por ejemplo, véase [Coleman+94, MO95, Fowler96]), un modelo de análisis sugiere esencialmente un modelo de objetos del dominio —un estudio y descripción de los conceptos del dominio—. Pero el “Modelo de Análisis” del UP es una primera versión del Modelo de Diseño del UP describiendo objetos del software que colaboran, con responsabilidades. Citando textualmente, “El modelo de análisis

es una abstracción, o generalización, del diseño” [Kruchten00]. Y, “Un modelo de análisis puede verse como un primer corte de un modelo de diseño” [JBR99].

El equipo del producto del RUP destaca que es opcional y que su valor no es frecuente, y no fomenta que se cree habitualmente —ya que es otro conjunto más de diagramas que hay que crear antes de la implementación, y los metodologistas y arquitectos expertos rara vez lo utilizan—.

37.8. El producto del RUP

El producto del RUP es un conjunto de documentación basada en el web cohesivo y bien diseñado (páginas HTML) vendido por Rational Software que describe el Proceso Unificado de Rational, un refinamiento actualizado y detallado del UP, que es más general. Describe todos los artefactos, actividades y roles, proporciona guías e incluye plantillas para la mayoría de los artefactos (ver Figura 37.4).

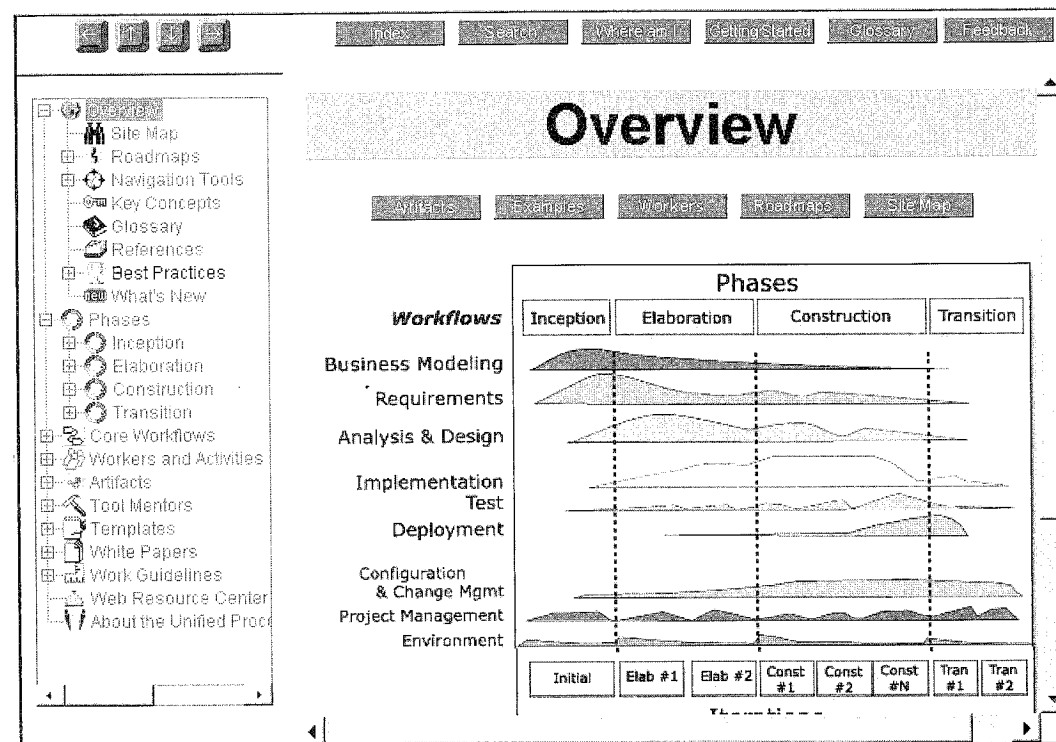


Figura 37.4. El producto del RUP.

El UP se puede aplicar o adoptar con la ayuda de tutores del proceso y libros; las ideas básicas, como el desarrollo iterativo, se describen en éste y en otros libros. En consecuencia, no es necesario poseer el producto del RUP. No obstante, algunas organizaciones consideran que colocar este producto basado en el web (y sus plantillas) en su intranet (respetando la licencia) en una localización visible es un mecanismo simple y efectivo para extender gradualmente su adopción. Que una organización pase a un nue-

vo proceso de desarrollo, más allá de un nivel superficial, requiere que se le apoye de varios modos. Además de tutores para el proceso, proyectos pilotos y seminarios, la documentación y las plantillas basadas en el web proporcionadas por el producto del RUP, sin duda son ayudas útiles que merece la pena que se evalúen.

37.9. Los desafíos y mitos de la reutilización

El UP está desarrollado con los proyectos de la tecnología de objetos (TO) en mente, y con frecuencia se promueve la adopción de la TO para conseguir la **reutilización** del software. Conseguir reutilizar de manera significativa es un objetivo loable, pero difícil. Es una función de mucho más que adoptar la TO y escribir clases; la TO no es más que una tecnología que la posibilita mediante un conjunto de cambios técnicos, organizativos y sociales que tienen que ocurrir para poder apreciar una reutilización significativa. Ciertamente, las librerías de clases para los servicios técnicos, como las librerías de la tecnología Java, proporcionan un gran ejemplo de reutilización, pero me estoy refiriendo a la dificultad de reutilizar código creado en una organización, no a las librerías básicas.

En una encuesta a organizaciones que han adoptado la TO, se les preguntó el valor real de su adopción. Curiosamente, la reutilización estaba *al final* de la lista [Cutter97]. Entre los expertos y organizaciones con experiencia en la TO, no es una sorpresa: saben que la popular descripción de la prensa de la TO para reutilizar es, en cierto modo, un mito; la mayoría de las organizaciones lo ven poco. Esto no implica que no sea un objetivo valioso, o que no haya reutilización —merece la pena, y ha habido algo—. Pero no los altos niveles de reutilización que sugieren muchos artículos y libros. Y muchos desarrolladores con experiencia en la TO puede contarle una historia de guerra sobre el intento desorientado a gran escala de una organización para crear las grandes “librerías reutilizables” o servicios para la compañía, gastando un año y millones de dólares, y terminando con un proyecto fracasado, o uno que perdió el rumbo. La reutilización es difícil, y se puede sostener que es una función de cuestiones más sociales y organizativas que técnicas.

¿Significa esto que la TO no tiene valor? En absoluto, pero su valor se ha asociado incorrectamente sobre todo con la reutilización, en lugar de al modo en el que ayuda de manera más perceptible en la práctica: flexibilidad, facilidad de cambio y manejo de la complejidad. La misma encuesta [Cutter97] lista los valores sobresalientes que realmente se experimentan al adoptar la TO: mantenimiento de la aplicación más simple y ahorro de costes. Los sistemas de objetos —si se diseñan bien— son relativamente más fáciles o más rápidos de modificar y extender, que si se utilizan tecnologías no orientadas a objetos. Esto es importante; muchas organizaciones encuentran que la mayoría de los costes globales a largo plazo de una aplicación tienen que ver con la revisión y el mantenimiento, no con el desarrollo original y, por tanto, son importantes las estrategias que permitan reducir los costes de revisión. Aunque es racional querer reducir los costes de desarrollo de nuevos sistemas, es irónico que pocas de las personas involucradas pregunten lo que sigue, “¿Cómo podemos reducir los costes de revisión y mantenimiento?” cuando con frecuencia es lo más caro. Es aquí donde puede contribuir la TO, además de su fuerza y elegancia al abordar sistemas complejos.

38.1. Notación general

Especificaciones de estereotipos y propiedades con etiquetas

Los estereotipos se utilizan en UML para clasificar un elemento (ver Figura 38.1).

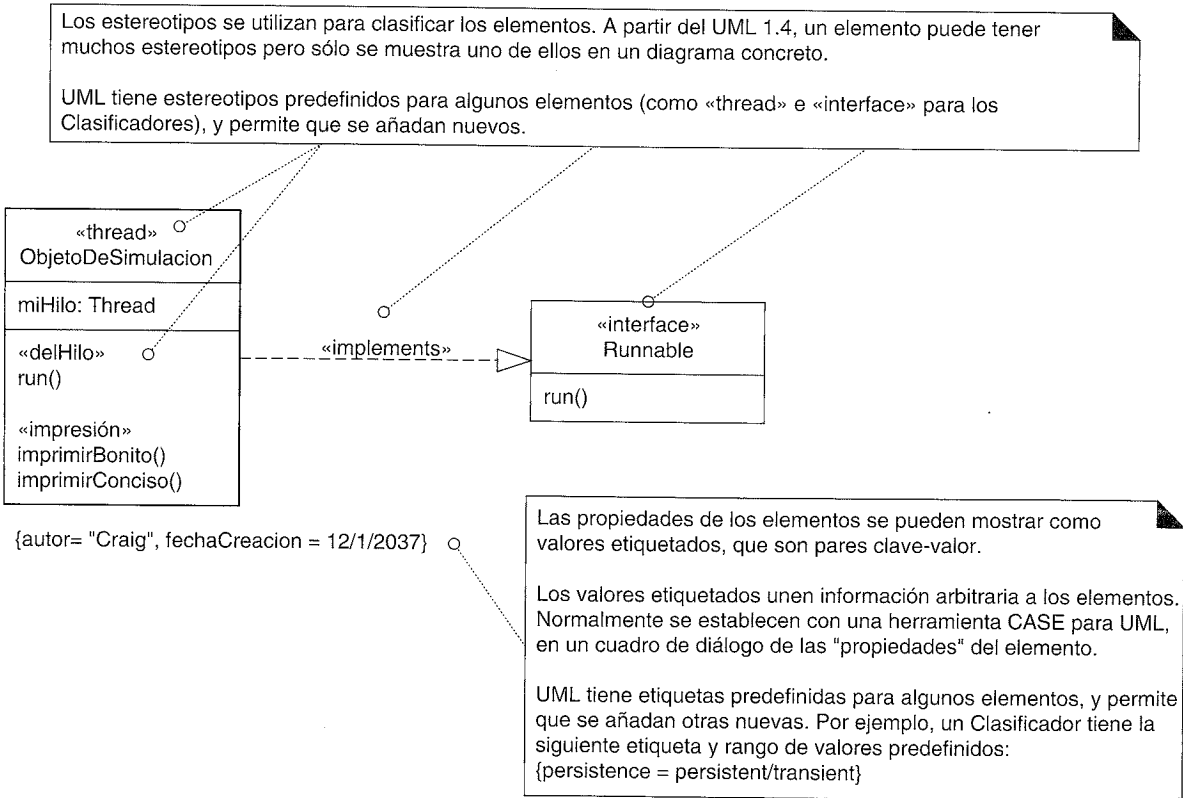


Figura 38.1. Estereotipos y propiedades.

Interfaces de paquetes

Se puede representar un paquete que implementa una interfaz (ver Figura 38.2).

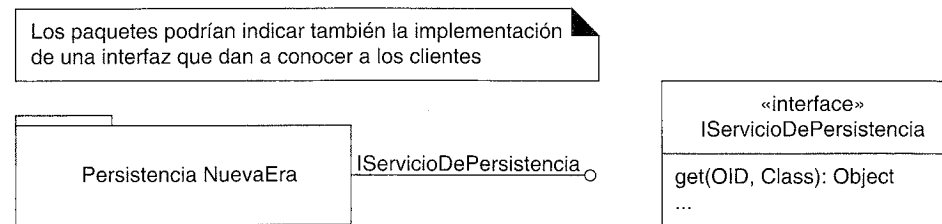


Figura 38.2. Interfaz de un paquete.

Dependencia

Pueden existir dependencias entre elementos cualesquiera, pero probablemente se utilizan con más frecuencia entre en los diagramas de paquetes de UML para ilustrar las dependencias de los paquetes (ver Figura 38.3).

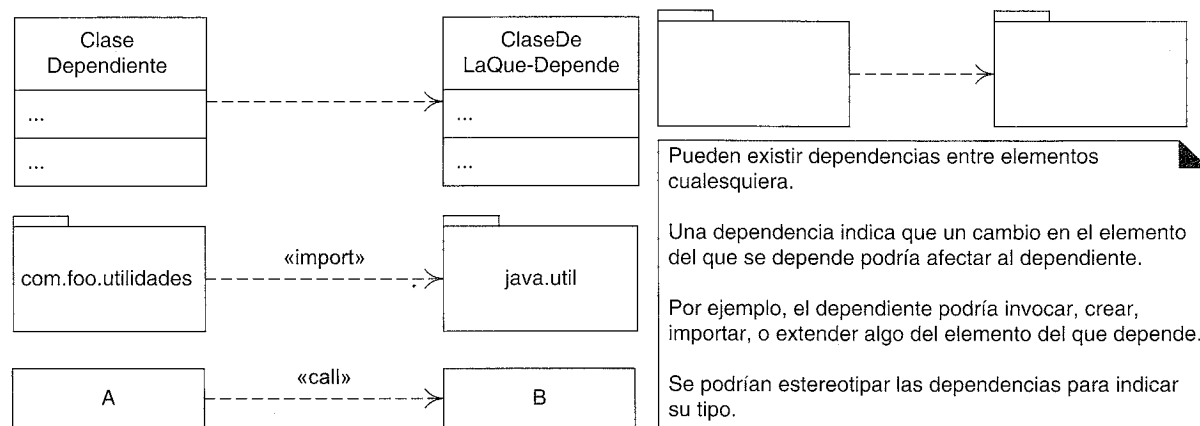


Figura 38.3. Dependencias.

38.2. Diagramas de implementación

UML define varios diagramas que se pueden utilizar para ilustrar los detalles de implementación. El que se utiliza más comúnmente es un diagrama de despliegue, para ilustrar el despliegue de los componentes y procesos en los nodos de proceso.

Diagramas de componentes

Citando textualmente: Un **componente** representa una parte de un sistema modular, desplegable, y reemplazable, que encapsula la implementación y expone un conjunto de interfaces [OMG01]. Podría ser, por ejemplo, código fuente, binario o ejecutable. Entre los

ejemplos encontramos navegadores o servidores HTTP, una base de datos, una DLL, o un fichero JAR (como para un Enterprise Java Bean). Los componentes en UML se representan en los diagramas de despliegue, en lugar de independientemente. La Figura 38.4 ilustra algo de la notación común.

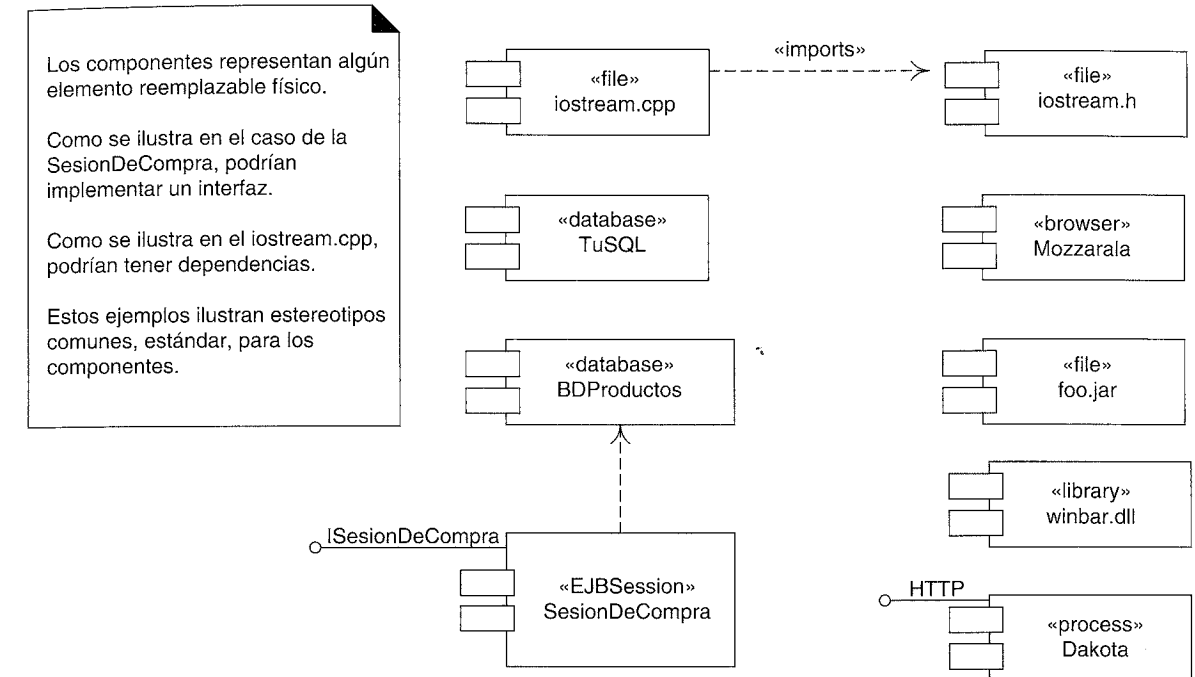


Figura 38.4. Componentes en UML.

Diagramas de despliegue

Un diagrama de despliegue muestra cómo se configuran las instancias de los componentes y los procesos para la ejecución run-time en las instancias de los **nodos** de proceso (algo con memoria y servicios de proceso; ver Figura 38.5).

38.3. Clase plantilla (parametrizada, genérica)

En la Figura 38.6 se muestran las clases plantilla y su instanciación.

Algunos lenguajes, como C++, soportan las clases plantilla, genéricas o parametrizadas. Además, esta característica se añadirá al lenguaje Java. Por ejemplo, en C++, *tabla<String, Persona>* declara la instanciación de la clase plantilla con claves de tipo *String*, y valores de tipos *Persona*.

38.4. Diagramas de actividades

Un **diagrama de actividades** de UML ofrece una notación rica para representar una secuencia de actividades. Podría aplicarse a cualquier propósito (como para mostrar los pasos de un algoritmo), pero se considera especialmente útil para visualizar los flujos de

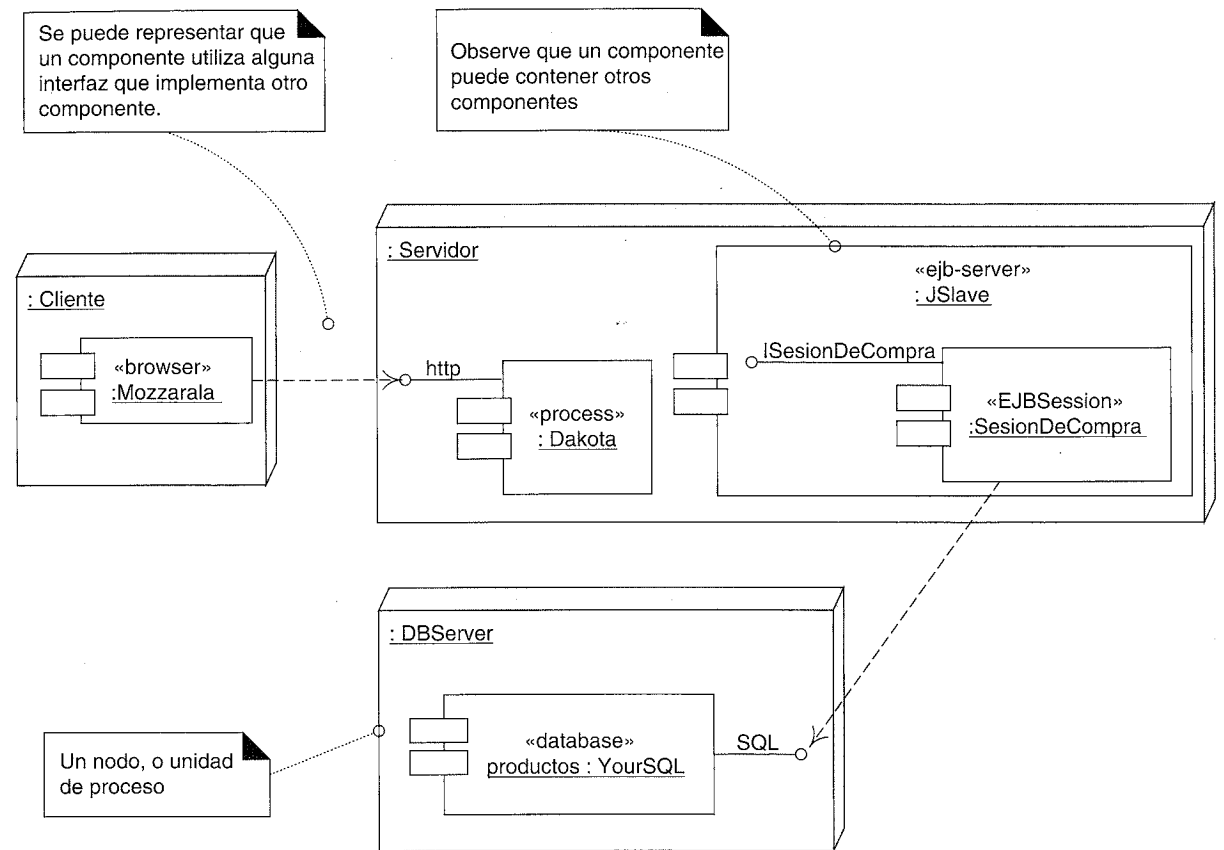


Figura 38.5. Un diagrama de despliegue.

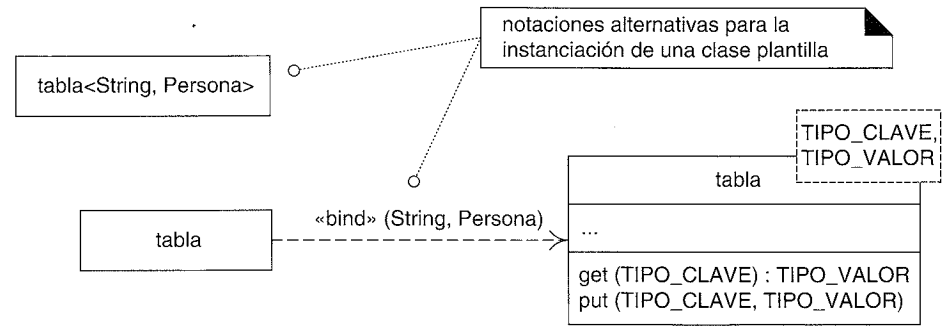


Figura 38.6. Clases plantilla.

trabajo y los procesos del negocio, o casos de uso. Uno de los flujos de trabajo (disciplinas) del UP es el **Modelado del Negocio**; su propósito es entender y comunicar “la estructura y la dinámica de la organización en el que se va a desplegar un sistema” [RUP]. Un artefacto clave de la disciplina del Modelado del Negocio es el **Modelo de Objetos del Negocio** (un superconjunto del Modelo del Dominio del UP), que visualiza esencialmente cómo funciona un negocio utilizando diagramas de clases, secuencia y actividades de UML. De esta manera, los diagramas de actividades se aplican especialmente dentro de la disciplina del Modelado del Negocio del UP.

Entre la notación destacada encontramos actividades concurrentes, calles y relaciones de flujo acción-objeto, como se ilustra en la Figura 38.7 (adaptado a partir de [OMG01, FS00]). Formalmente, un diagrama de actividad se considera un tipo especial de diagrama de estados de UML en el que los estados son acciones, y las transiciones de los eventos se disparan automáticamente al completarse la acción.

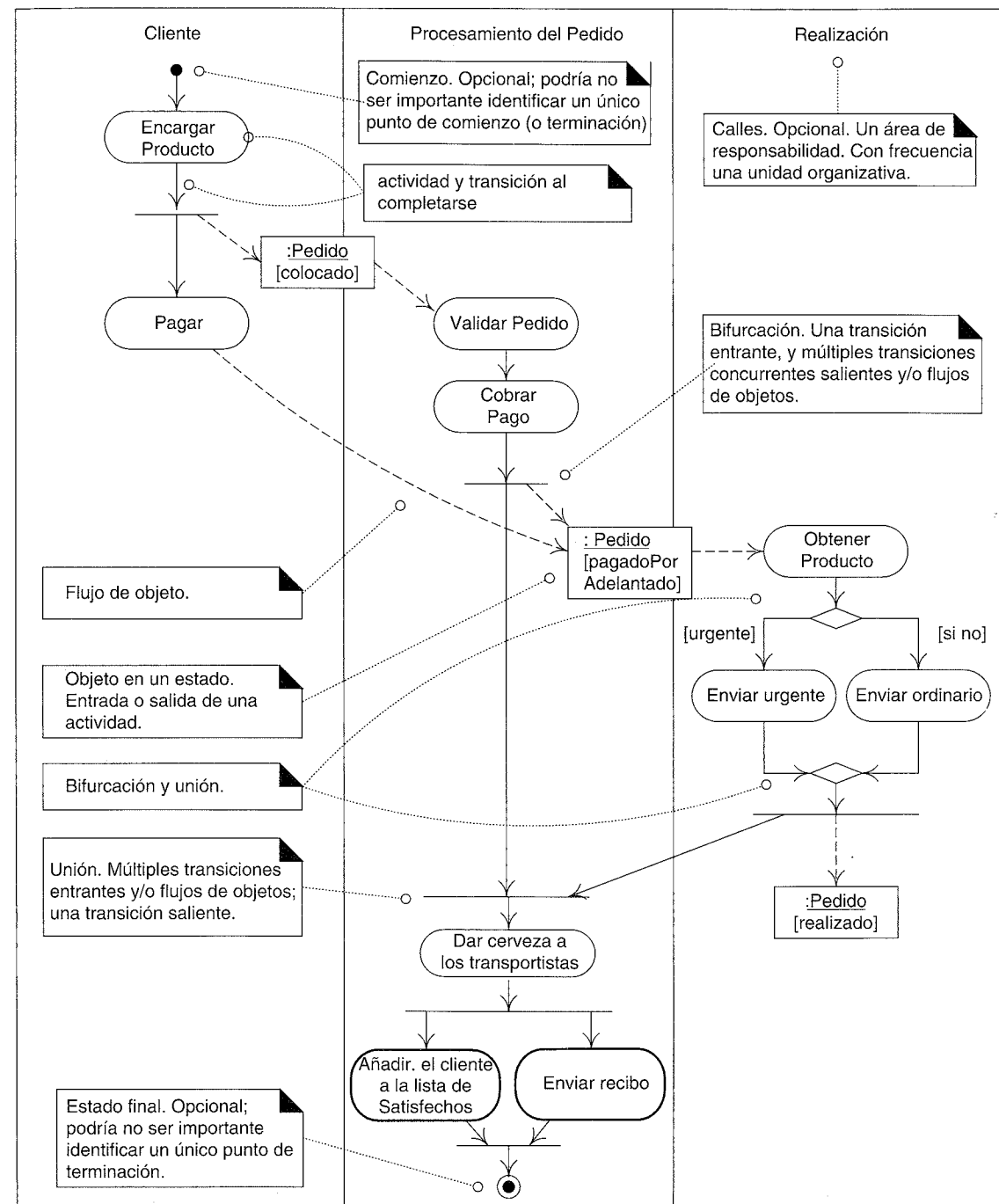


Figura 38.7. Diagrama de actividades.